# Building Industrial RAG Systems from Daily Drilling Reports

**A hands-on guide to retrieval-augmented engineering intelligence**

Stephane Djimra

July 2026

# Table of contents

# Welcome

Every rig produces a paper trail. Every well produces a story — told in Daily Drilling Reports (DDRs), scattered across shared drives, scanned PDFs, and email attachments, written by dozens of different hands in a shorthand that only the field understands.

That story holds answers your team needs today: *Have we seen this stuck pipe signature before? What usually precedes a screenout on this formation? Which offset wells lost circulation at this depth?*

Somewhere in your archive, the answer already exists. Finding it is the problem.

This book builds, from scratch and in plain Python, a system that can read that archive and answer those questions — with evidence, not guesses. You will not start with theory. You will start with a single PDF and a script that reads it. By the end, you will have an Industrial DDR Intelligence Platform: a retrieval-augmented generation (RAG) system purpose-built for drilling and completions data, that you built chapter by chapter and understand end to end.

Every example in this book uses real Daily Drilling Reports — not invented ones. The archive is from **Utah FORGE**, a Department of Energy-funded geothermal research well (FORGE 16A(78)-32) whose reports are publicly available. No anonymisation, no synthetic stand-ins: you'll read genuine rig-floor language, including a real stuck-pipe event and a real fishing operation, from page one.

## Who this book is for

You should read this book if you are a drilling engineer, completions engineer, intervention engineer, production engineer, petroleum data scientist, or part of a digital transformation team in energy, and you:

- Have basic Python knowledge (variables, functions, loops, lists/dicts).
- Have never built a RAG system.
- Have never used a vector database or worked with embeddings.
- Want working tools, not a machine learning course.

No prior AI/ML background is assumed. Every concept is introduced exactly when you need it to solve the problem in front of you — not before.

## What you will build

| Chapter | You will be able to… |
|---|---|
| 1 | Extract text from a DDR PDF |
| 2 | Expand oilfield abbreviations (BHA, WOB, MWD…) automatically |
| 3 | Keyword-search your entire DDR archive |
| 4 | Search DDRs by meaning, not just exact words |
| 5 | Ask a question and get a cited, evidence-backed answer |
| 6–12 | Scale the system to scanned reports, large archives, hybrid retrieval, reranking, and full traceability |

## How this book works

Every chapter solves one operational problem and ends with something you can run. There is no chapter you read passively — you will type code, run it against real DDR text, and see it work. Theory appears only when it explains *why* the code in front of you behaves the way it does.

We follow one rule throughout: **engineering usefulness over AI novelty**. Retrieval matters more than generation. Traceability matters more than eloquent prose. Engineers trust evidence, not predictions — so this system is built to show its work, every time.

## Companion repository

All code, sample datasets, and notebooks referenced in this book live in the companion repository. See Appendix A for setup instructions. Every chapter lists exactly which files you need under **Repository files**.

Let's read our first DDR.

# Preface

## Why this book exists

I wrote this book because I kept seeing the same gap on drilling and completions teams: the field generates enormous amounts of written operational knowledge — Daily Drilling Reports, end-of-well reports, morning reports, NPT logs — and almost none of it is *usable* by the next engineer who needs it (Society of Petroleum Engineers 2022). It sits in PDF form, scanned or digital, named inconsistently, scattered across SharePoint folders and email threads. When a rig hits a screenout at 9,200 ft, the fastest way to find out if this has happened before is usually to ask the oldest person in the room.

Large language models and retrieval-augmented generation (RAG) are genuinely useful here — not because they are new or exciting, but because they solve a real, boring, expensive problem: turning an unstructured archive into something you can query. This book teaches you to build that system yourself, understand every part of it, and trust its output, rather than adopting a black-box product you cannot inspect.

## Why "industrial"

A demo that answers one question about one PDF is easy to build and easy to find online. An industrial system is different. It has to:

- Handle scanned reports as well as digital ones.
- Scale to thousands of DDRs across dozens of wells.
- Never silently invent an answer — every claim must trace back to a specific report and page.
- Be auditable by someone who was not in the room when it was built.

Every chapter in Part II exists because a toy RAG system fails at one of these requirements. We fix them one at a time, in the order you would actually hit them in the field.

## How to read this book

This book follows the practical-first, project-based teaching style of *Python Crash Course* (Matthes 2023): small wins early, theory only when the code in front of you needs it, and a working artifact at the end of every chapter. Read it in order the first time. Each chapter assumes the code from the previous one exists and works — this is a build, not a reference manual. After your first pass, use it as a reference: jump to the chapter that matches the problem you currently have (Chapter 6 if your PDFs are scanned, Chapter 9 if your retrieval quality is poor, Chapter 10 if your answers aren't trustworthy yet).

Type the code yourself. Do not copy-paste from the companion repository until you have typed it once and watched it run. This is slower and it is worth it.

## A note on data

Most Daily Drilling Reports are operationally sensitive and confidential — which is exactly why this book doesn't use one from a commercial operator. Every example here comes from **Utah FORGE**, a Department of Energy-funded enhanced geothermal system research well (FORGE 16A(78)-32) whose reports are public: real well name, real dates, real personnel, real events, exactly as filed. No anonymisation was applied, because none was needed — this is what "publicly available" means in practice, and it lets this book show you genuine field language instead of an approximation of it.

That said, the *technique* generalises directly to a confidential archive: everything from Chapter 6 onward — OCR gating, traceability, corpus-completeness checks — exists precisely because production archives are messier and more sensitive than this one. If you point this book's code at your own organisation's DDRs, do not commit them to a public repository, and see Appendix A for a data-handling checklist.

# Acknowledgements

This book exists because of the drilling and completions engineers who patiently explained, over many years and many wells, why the report says "POOH to change BHA due to erratic torque" and what that sentence actually means for the next twelve hours of operations. Domain knowledge like that does not live in a textbook — it lives in people, and in the daily reports they write under time pressure at the end of a twelve-hour tour.

Thanks are due to the open-source communities behind the tools this book relies on throughout: Python, Quarto, Jupyter, pandas, and the broader Python data and NLP ecosystem. None of the engineering in this book would be reproducible without them.

Finally, thanks to every reader working through this material on a real archive of real reports. If you find an error, an oilfield abbreviation that's wrong for your basin, or a chapter that assumes something it shouldn't, please open an issue in the companion repository — corrections from practitioners are what keep a book like this honest.

# Part I.

# Part I — Foundations

# 1. Reading Your First DDR

## 1.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain why DDR PDFs are hard for software to read, and the difference between a *digital-native* PDF and a *scanned* one.
- Extract raw text from a digital-native PDF using Python.
- Write a script that processes a single DDR, or an entire folder of them, from the command line.
- Save extracted text so it's ready for the cleaning work in Chapter 2.

## 1.2. Operational problem

This book uses a real, publicly available archive: the Daily Drilling Reports from **Utah FORGE** — a Department of Energy-funded research project at the University of Utah, drilling and testing an enhanced geothermal system (EGS) well, **FORGE 16A(78)-32**, in Beaver County, Utah. Because it's a public research well rather than a commercial operator's confidential asset, its DDRs are openly available — which means every example in this book is real, traceable text, not an invented stand-in.

Picture the same folder problem any operations team faces: 76 DDR PDFs, one per day of drilling and completion work, named inconsistently by whatever software produced them. Somewhere in that folder is the report that matters most this week — the day the drill string got stuck, or the day a fishing run recovered lost bit pieces. Opening 76 PDFs one at a time to find the right paragraph is exactly the kind of task a short Python script should do for you.

That's the whole job of this chapter: given one PDF, get its text into a Python string you can work with. Nothing clever yet — no search, no abbreviation handling, no AI. Just a reliable way to turn a PDF into text. Everything in this book is built on top of getting this first step right.

## 1.3. Example DDR extract

Real DDR excerpt — report #38, FORGE 16A(78)-32, 2020-11-26

```
RPT DATE:11/26/2020
DAILY DRILLING REPORT RPT NUM.:38
WELL NAME:FORGE 16A [78]-32 JOB:Drilling 65° Tangent
PRESENT OPERATIONS:PIPE FREE, TRIP OUT OF HOLE FOR BHA INSPECTION

TIME BREAKDOWN
23:30 04:00 4.5 Drill From 6,360' to 6,507', (147') Total, 32.6' feet per hour.
WOB 20 TO 35k, Rotary 50, Torque 6,500, SPP 3200-3400 GPM 560, DIFF 200-
300psi
During the slide lost tool face and became assembly became stuck
04:00 06:00 2.0 Work pipe, circulate lube sweep, work tool back in position
Pipe free
```

This is one of ten real reports curated for Part I, spanning the well's actual timeline from rig-up in October 2020 through this stuck-pipe day in November and a fishing operation in December. Every excerpt in Part I is genuine, unedited report text — no anonymisation was needed, because this data is already public.

## 1.4. Theory

A PDF is not a text file wearing a costume. It's a page-description format: instructions for *drawing* characters at specific coordinates, not a stream of text with a beginning and an end. Two DDRs that look identical on screen can be built completely differently under the hood:

- **Digital-native PDFs** were generated directly from software (in this case, WellEz, the drilling data system that produced every report in this archive). The text was placed on the page as actual character data, so a library can recover it.
- **Scanned PDFs** are photographs of paper reports. There is no character data at all — just a picture of a page. Extracting text from these requires optical character recognition (OCR), which we cover in Chapter 6.

The Utah FORGE archive is entirely digital-native — every report was produced by the same software, so extraction is reliable throughout Part I. This won't always be true of a real archive (Chapter 6 deals with the scanned-report case), but it's the right place to start.

We'll use `pdfplumber`, a Python library that reads a PDF's internal structure and hands back the text (and, later in the book, tables and layout) on each page. There are other libraries that do this — `PyPDF2`, `pymupdf` — and they're worth knowing about, but `pdfplumber` is a good default: actively maintained, and its text extraction is reliable on the kind of structured reports we're working with.

```
DDR PDF
```

```
   ↓
open with pdfplumber
   ↓
for each page:
   ↓
  page.extract_text()  -->  string, or None if unextractable
   ↓
join every page's text together
   ↓
one Python string, ready for Chapter 2
```

## 1.5. Implementation

### 1.5.1. Step 1: get the sample archive

The ten curated reports used throughout Part I are already in `datasets/sample_ddrs/` in this repository — real PDFs, committed directly, since Utah FORGE data is public. You don't need to generate or download anything to start.

```
import pathlib
sorted(pathlib.Path("datasets/sample_ddrs").glob("*.pdf"))
```

If you want to build this curated subset yourself from a full local copy of the public archive (or extend it with more reports), see `code/chapter_01/build_sample_archive.py` — it's documented in Appendix A.

### 1.5.2. Step 2: extract text from one PDF

Here is the core of `code/chapter_01/read_ddr.py`, built up piece by piece.

First, open a PDF and read one page:

```
import pdfplumber

with pdfplumber.open("datasets/sample_ddrs/FORGE-16A-78-32_Drilling_038_2020-11-26.pdf") as pdf:
    first_page = pdf.pages[0]
    print(first_page.extract_text())
```

Run that and you'll see the report's text printed straight to your terminal — the full header block (well name, spud date, days from spud), casing and mud tables, the BHA component list, survey data, and the time breakdown — all as a plain Python string. Real DDRs pack in far more than a simple narrative: this one report alone has seven distinct data sections.

Every DDR in this archive is a single page, but that won't be true of every report you'll ever encounter. Loop over every page and keep track of which page each chunk of text came from — you'll need that later for citations:

```python
from pathlib import Path
import pdfplumber

def extract_text(pdf_path: Path) -> str:
    pages_text = []
    with pdfplumber.open(pdf_path) as pdf:
        for page_number, page in enumerate(pdf.pages, start=1):
            text = page.extract_text() or ""
            pages_text.append(f"--- Page {page_number} ---\n{text}")
    return "\n\n".join(pages_text)
```

Note the `or ""`. If a page has no extractable text — a scanned page mixed into an otherwise digital report, for example — `extract_text()` returns `None`, and every downstream step in this book expects a string, not `None`. This one line is a small example of a pattern you'll see throughout the book: assume real data is messier than the happy path.

### 1.5.3. Step 3: make it a script you can run from the command line

The full script, `code/chapter_01/read_ddr.py`, wraps this in a command-line interface so you can point it at one file or a whole folder:

```bash
# One file, printed to the terminal
python code/chapter_01/read_ddr.py datasets/sample_ddrs/FORGE-16A-78-32_Drilling_038_2020-11-2

# A whole folder, saved as .txt files
python code/chapter_01/read_ddr.py datasets/sample_ddrs/ --batch --out datasets/ddr_text
```

Try the first command now. You should see the full report text, including the line: `During th e slide lost tool face and became assembly became stuck` — real language from a real rig report, not a paraphrase.

## 1.6. Practical exercise

**Try it yourself:** Run `read_ddr.py` in batch mode against `datasets/sample_ddrs/`, saving the output to `datasets/ddr_text/`. Then open `FORGE-16A-78-32_Drilling_050_2020-12-08.txt` and find the line describing what the crew milled up that day.

**You'll know it worked when:** you have ten `.txt` files in `datasets/ddr_text/`, one per PDF, and you can find the word "Fishing" in report 050's text without opening the original PDF.

## 1.7. Field notes

⚠ Field notes: a table that reads like nonsense until you know why

**Action:** extract report #38's `BHA` section and read it as plain text.
**Result:**

```
# COMPONENT OD ID LENGTH # COMPONENT OD ID LENGTH
1 Drill Bit-Reed SKC613M-01C 8.75 1 7 Monel Collar-NMDC with MWD 6.75 3.25 30.35
2 Motor-6.5'' 7/8, 5.7, 0.242 RPG 1.50,* Fixed 6.5 33.01 8 Monel Collar-
NMDC (2 joints) 6.75 3.25 60.66
```

**Why:** the real BHA table on this report is laid out as **two side-by-side sub-tables** — components 1–6 on the left half of the page, components 7–10 on the right half. Visually, that's obvious: two neat columns. But `extract_text()` reads left to right, top to bottom, so component 1's row and component 7's row — which just happen to sit on the same horizontal line of the page — get concatenated into one line of output text, with no marker showing where the left sub-table ends and the right one begins.
**Lesson:** `extract_text()` gives you the words on the page, not the *layout* those words depend on to make sense. A two-column table isn't a special case — it's normal DDR formatting — and reading its flattened output at face value would have you believe component 1 and component 7 are the same row. This book's chapters work around it by favouring the narrative `TIME BREAKDOWN` text over the structured tables for search and retrieval — real table-aware parsing (detecting table boundaries and reconstructing rows correctly) is its own discipline, and the companion pipeline's `src/rag_pdf/table_detect.py` and `table_extract.py` handle it properly. For now, just know that "extracted text" and "correct reading order" are not the same guarantee.

## 1.8. Challenge exercise

**Challenge:** Extend the script to also print, for each PDF, the total number of pages and the total character count extracted. Then compare report #3 (October 22, before the well was spudded — `PRESENT OPERATIONS: RIGGING UP...`) against report #38 (the stuck-pipe day): how much richer is the later report's text, and why might that be? (Hint: check the `SPUD DATE` and `DFS/DOL` fields in each report's header.) A reference solution is in `code/chapter_01/challenge/`.

## 1.9. Key takeaways

- A PDF's internal structure, not its on-screen appearance, determines whether text can be extracted directly. Digital-native PDFs can; scanned PDFs need OCR (Chapter 6).
- `pdfplumber` turns a PDF into plain Python strings, one page at a time.
- Handle `None` from `extract_text()` explicitly — real-world DDR archives will contain pages that don't extract cleanly, and your code should never crash on them.

- A five-line function (`extract_text`) plus a thin command-line wrapper is already a genuinely useful tool. You don't need machine learning to save an engineer an hour of searching PDFs by hand — and it works identically whether the PDF is a synthetic example or, as here, a real report from a real well.

## 1.10. Repository files

| File | Purpose |
|---|---|
| `code/chapter_01/read_ddr.py` | Extracts text from one PDF or a folder of PDFs |
| `code/chapter_01/build_sample_archive.py` | Reproduces the curated Part I subset from the full public archive |
| `datasets/sample_ddrs/` | Ten real, curated Utah FORGE DDR PDFs used throughout Part I |
| `notebooks/chapter_01_explore.ipynb` | Interactive notebook version of this chapter |

## 1.11. Suggested next step

The text you just extracted uses real oilfield shorthand and drilling terminology — `BHA`, `WOB`, `SPP`, `PJSM` — some spelled out, some abbreviated, exactly as the field actually writes it. Chapter 2 builds an abbreviation expansion engine grounded in what's actually in this archive, turning this raw text into something both humans and machines can search reliably.

# 2. Cleaning Operational Text

## 2.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain why oilfield shorthand breaks naive text search and NLP.
- Build a dictionary-based abbreviation expander that respects word boundaries so it doesn't corrupt unrelated text.
- Apply the expander across every `.txt` file produced in Chapter 1.

## 2.2. Operational problem

Open one of the `.txt` files you extracted in Chapter 1 and read it as a computer would. Report #38's time breakdown says: *"During the slide lost tool face and became assembly became stuck"* — plain enough. But the same report is full of terms a keyword search or language model has no way to interpret: `BHA`, `WOB`, `MWD`, `SPP`, `NMDC`, `DLS`, `ECD`. A drilling engineer reads these instantly. A keyword search for "bottom hole assembly" finds nothing in a report that only ever writes `BHA`. Before we can search, summarise, or embed this text well, we need to expand it into language that means the same thing to a human and to a machine.

## 2.3. Example DDR extract

Real excerpt — report #38, showing both patterns

```
PJSM, pre job safety meeting.                    <- self-
expanded by the source
...
Pick up Curve assembly 2, BHA 21, Reed Hycalog     <- BHA never expanded
SKC613M-01C Trip in hole to 5,200'
...
WOB 20 TO 35k, Rotary 50, Torque 6,500, SPP        <- WOB, SPP never expanded
3200-3400 GPM 560, DIFF 200-300psi
```

## 2.4. Theory

An abbreviation expander is just a dictionary lookup — `{"BHA": "bottom hole assembly", "WOB": "weight on bit", ...}` — but two details make or break it:

1. **Word boundaries.** `str.replace("MD", ...)` will happily mangle a word that merely *contains* "MD" as a substring. Match on whole words using regex `\b...\b`, not raw substring replacement.
2. **Case.** DDRs mix cases inconsistently — this archive's headers are often all-caps while narrative text is mixed-case — so expansion needs to be case-insensitive without forcing the rest of the sentence to change case.

This is deliberately not a machine-learning problem. A few dozen well-chosen abbreviations, expanded correctly, solve most of the readability problem — and unlike a model, a dictionary is auditable: you can list every expansion it will ever make.

```
raw text
   ↓
for each (abbreviation, expansion) in dictionary:
   ↓
  build pattern: \b + abbreviation + \b   (case-insensitive)
   ↓
  substitute every match with the expansion
   ↓
expanded text
```

## 2.5. Implementation

```python
# code/chapter_02/expand_abbreviations.py
import re

ABBREVIATIONS = {
    "BHA": "bottom hole assembly",
    "WOB": "weight on bit",
    "RPM": "revolutions per minute",
    "ROP": "rate of penetration",
    "MD": "measured depth",
    "TVD": "true vertical depth",
    "SPP": "stand pipe pressure",
    "GPM": "gallons per minute",
    "DLS": "dogleg severity",
    "MWD": "measurement while drilling",
    "NMDC": "non-magnetic drill collar",
    "UBHO": "universal bottom hole orientation sub",
    "NPT": "non-productive time",
```

```
    "ECD": "equivalent circulating density",
    "MW": "mud weight",
    "DFS": "days from spud",
    "DOL": "days on location",
    "PDC": "polycrystalline diamond compact",
}

def expand_text(text: str, abbreviations: dict[str, str] = ABBREVIATIONS) -> str:
    for abbr, expansion in abbreviations.items():
        pattern = r"\b" + re.escape(abbr) + r"\b"
        text = re.sub(pattern, expansion, text, flags=re.IGNORECASE)
    return text
```

Run it against the text files from Chapter 1's `datasets/ddr_text/` folder and save the expanded versions to `datasets/ddr_text_expanded/`.

## 2.6. Practical exercise

**Try it yourself:** Run the expander against `FORGE-16A-78-32_Drilling_038_2020-11-26.txt`.

**You'll know it worked when:** `Pick up Curve assembly 2, BHA 21` becomes `Pick up Curve assembly 2, bottom hole assembly 21`, and no unrelated word (check the `MD`/`TVD` survey table carefully — short abbreviations are the ones most likely to collide with real words) gets corrupted.

## 2.7. Field notes

⚠ Field notes: the source already expands some abbreviations — but not the ones that matter

**Action:** search every report for the literal string `PJSM`.
**Result:** it appears in all ten sample reports — and every single time, it's immediately followed by its own expansion, written by the source software itself: `PJSM, pre job safety meeting`.
**Why:** this archive's report-generation software (WellEz) evidently has a template that spells "pre job safety meeting" out in full every time, right after the abbreviation. But run the same check against `BHA`, `WOB`, `MWD`, or `SPP` — the terms that actually carry operational meaning in the time breakdown — and none of them ever get that treatment, anywhere in the archive.
**Lesson:** don't assume a real archive is consistently either abbreviated or expanded. This one is both, unevenly, by accident of whatever template a report-writing tool happened to use for one specific phrase. An automated expander has to cover every term regardless — you can't rely on the source to have already done the job for the ones that matter.

## 2.8. Challenge exercise

**Challenge:** Extend `ABBREVIATIONS` using Appendix B, and handle the mud-table abbreviations (`FV`, `WL`, `PV`, `YP`, `CL`) — notice how short these are, and think carefully about whether blindly expanding a two-letter token like `PV` is actually safe against this archive's real text, or whether it needs a narrower match context. A reference solution is in `code/chapter_02/challenge/`.

## 2.9. Key takeaways

- Word-boundary regex, not substring replacement, is the difference between a correct expander and a silently broken one.
- Real archives are inconsistent by nature — the same report can spell one abbreviation out in full while never expanding another. Don't assume the source data will document itself.
- A transparent dictionary beats an opaque model here: every expansion is inspectable and testable.
- Cleaning text is not optional groundwork — it's what makes every later chapter's search and retrieval actually work.

## 2.10. Repository files

| File | Purpose |
|---|---|
| `code/chapter_02/expand_abbreviations.py` | Word-boundary abbreviation expander |
| `datasets/ddr_text_expanded/` | Expanded output from Chapter 1's extracted text |
| `notebooks/chapter_02_explore.ipynb` | Interactive companion notebook |

## 2.11. Suggested next step

With clean, expanded text in hand, Chapter 3 builds the first tool an engineer would actually reach for: a keyword search engine that answers "which reports mention losses?" in milliseconds.

# 3. Searching DDRs

## 3.1. Learning objectives

By the end of this chapter, you will be able to:

- Build an inverted index over a folder of cleaned DDR text.
- Answer keyword queries like "which reports mention losses?" in milliseconds instead of minutes.
- Explain why exact keyword matching, while fast and simple, will eventually miss things a drilling engineer would consider an obvious hit.

## 3.2. Operational problem

You now have clean, expanded text for every DDR. The next question is the one an engineer actually asks out loud: *"Which reports mention losses?"* or *"Show me the report where they had to fish for something."* Opening files one by one doesn't scale past a handful of reports — you need an index.

## 3.3. Example DDR extract

> **i** Real query and expected hit

```
Query: "losses"
Expected hit: FORGE-16A-78-32_Drilling_019_2020-11-07.txt
  → "Mud fluids seepage losses in six hours 9 bbls"
```

That's report #19 — a minor, real seepage-loss event, easy to miss by eye across 76 reports, trivial to find once indexed.

## 3.4. Theory

An **inverted index** flips the natural document → words relationship around: instead of asking "what words are in this document," it stores, for every word, *which documents contain it*. Looking up "losses" becomes a single dictionary lookup instead of scanning every file. This is the same core idea behind every search engine, from `grep` to Google — the difference is scale and ranking sophistication, not the fundamental mechanism.

Tokenization matters here: "losses" and "LOSSES" should be the same token, and punctuation shouldn't create spurious tokens (`6,507'` should not prevent `6507` or nearby words from matching cleanly).

```
documents                          inverted index
-------------------------------    -------------------------
report_038: "...became stuck..."   "stuck"   -> {report_038}
report_019: "...seepage losses..." "pipe"    -> {report_038}
report_050: "...milled up lost..." "losses"  -> {report_019}
                                    "milled"  -> {report_050}

query "stuck" -> look up "stuck" -> {report_038}   (one dict lookup,
                                                      not ten file scans)
```

## 3.5. Implementation

```python
# code/chapter_03/keyword_search.py
import re
from collections import defaultdict
from pathlib import Path

def tokenize(text: str) -> list[str]:
    return re.findall(r"[a-z0-9]+", text.lower())

def build_index(text_dir: Path) -> dict[str, set[str]]:
    index: dict[str, set[str]] = defaultdict(set)
    for path in sorted(text_dir.glob("*.txt")):
        tokens = set(tokenize(path.read_text(encoding="utf-8")))
        for token in tokens:
            index[token].add(path.name)
    return index

def search(index: dict[str, set[str]], query: str) -> set[str]:
    query_tokens = tokenize(query)
    if not query_tokens:
        return set()
    results = index.get(query_tokens[0], set()).copy()
    for token in query_tokens[1:]:
        results &= index.get(token, set())
    return results
```

`search()` here implements an AND query: a document must contain every query token to match — appropriate for a first version, and the challenge exercise asks you to add OR and phrase queries.

## 3.6. Practical exercise

**Try it yourself:** Build the index over `datasets/ddr_text_expanded/` and run `search(index, "fishing")`.

**You'll know it worked when:** the result set contains exactly `{"FORGE-16A-78-32_Drilling_050_2020-12-08.txt"}` — the day the crew milled up lost pieces of bit, right after packers failed to set the day before.

## 3.7. Field notes

⚠ Field notes: the search that misses the day after

**Query:** `"stuck pipe"`
**Result:** one hit — report #38 (2020-11-26), the day the assembly actually got stuck. Report #39, the very next day, is invisible to this query.
**Why:** report #39's own words are:

- "Work tight hole at 6,526'."
- "Due to high torque decision to pull out of hole"
- "Hole drag from 6,050' to 5,901' no issues"

but the word "stuck" never appears anywhere in report #39. `pipe` does (buried in a BHA table row, `Drill Pipe-5'' HWDP`), but the AND search still fails because `stuck` alone has zero matches outside report #38.
**Lesson:** an engineer researching *"how did the stuck-pipe situation on this well develop"* would want report #39 too — tight hole, high torque, and a decision to pull out of hole are exactly the kind of continued-risk language that matters the day after an incident. Keyword search can't surface it, because report #39 never uses the word the query asked for. This isn't a bug in the code above — it's the fundamental limit of matching on exact words at all. Chapter 4 comes back to this same query.

## 3.8. Challenge exercise

**Challenge:** Add phrase search (query tokens must appear *adjacent* and in order, not just co-occur anywhere in the document) and an OR operator. Test `search(index, "stuck")` against report #38, then try `"packers OR fishing"` and confirm it returns both report #49 (packers) and #50 (fishing). A reference solution is in `code/chapter_03/challenge/`.

## 3.9. Key takeaways

- An inverted index turns "search every file" into "look up a word," which is the difference between milliseconds and minutes at scale.

- Tokenization choices (case folding, punctuation handling) determine what counts as a match — get this wrong and searches silently fail.
- Exact keyword matching is fast, simple, and fundamentally limited: it cannot find report #49 ("packers did not set… pick up Fishing BHA") when you search "lost circulation," even though report #19's "seepage losses" line and report #49's packer failure are both, in a loose sense, "things going wrong downhole" — a human would connect them faster than exact keywords can.

## 3.10. Repository files

| File | Purpose |
|---|---|
| `code/chapter_03/keyword_search.py` | Inverted index and AND-query search |
| `notebooks/chapter_03_explore.ipynb` | Interactive companion notebook |

## 3.11. Suggested next step

Keyword search is exact and unforgiving — it doesn't know that "high torque decision to pull out of hole" and "stuck pipe" describe related trouble. Chapter 4 replaces exact matching with semantic search, so retrieval works by meaning, not just spelling — and comes back to this exact "stuck pipe" query to see how much that actually fixes.

# 4. Semantic Search

## 4.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain, without heavy math, what a text embedding is and why nearby vectors mean similar meaning.
- Generate embeddings for a folder of DDR text using `sentence-transformers`.
- Retrieve the most semantically similar chunks to a query using cosine similarity, without any exact keyword overlap required.
- Recognise that semantic search over whole documents is an improvement, not a complete fix — and that granularity is what actually determines how well it works.

## 4.2. Operational problem

Chapter 3 ended on a real miss: the query `"stuck pipe"` found report #38 (the actual incident) but was blind to report #39, which describes tight hole, high torque, and a decision to pull out of hole — real continued-risk language, written the very next day — without ever using the word "stuck." Semantic search is supposed to retrieve by *meaning* rather than exact words. Let's find out, honestly, how much of that problem it actually solves.

## 4.3. Example DDR extract

> **i** The same query, now by meaning

```
Query: "stuck pipe"  (same query Chapter 3 used)

Keyword search (Chapter 3): report #39 not in results at all.

Semantic search, whole-report embeddings (this chapter): report #39
  ranks 5th out of 10 - findable, but not prominent. Report #38 (the
  actual incident) ranks 2nd.
```

## 4.4. Theory

An **embedding model** converts a piece of text into a fixed-length vector of numbers — typically a few hundred dimensions — such that texts with similar meaning end up as vectors that are close together in that space. You don't need to understand the neural network that produces this vector; you need to understand what it gives you: a numeric representation of meaning you can compare with simple arithmetic.

**Cosine similarity** measures the angle between two vectors — 1.0 for identical direction, 0 for unrelated, negative for opposite. It's the standard way to compare embeddings because it ignores vector *length* and focuses purely on *direction*, which is what carries the meaning here.

We use `sentence-transformers` (Reimers and Gurevych 2019) with a small, fast model (`all-MiniLM-L6-v2`) — good enough to prove the concept and small enough to run on a laptop CPU. Chapter 8 replaces the brute-force search here with a proper vector database once the corpus grows past what fits comfortably in memory.

```
text ("stuck pipe")
   ↓
embedding model (all-MiniLM-L6-v2)
   ↓
vector: [0.02, -0.15, 0.33, ...]   (384 numbers)
   ↓
compare against every document's vector via cosine similarity
   ↓
nearby vectors = similar meaning, regardless of shared vocabulary
```

## 4.5. Implementation

```python
# code/chapter_04/semantic_search.py
from pathlib import Path

import numpy as np
from sentence_transformers import SentenceTransformer

MODEL_NAME = "all-MiniLM-L6-v2"

def load_chunks(text_dir: Path) -> tuple[list[str], list[str]]:
    filenames, texts = [], []
    for path in sorted(text_dir.glob("*.txt")):
        filenames.append(path.name)
        texts.append(path.read_text(encoding="utf-8"))
    return filenames, texts

def embed_texts(model: SentenceTransformer, texts: list[str]) -> np.ndarray:
```

```
    embeddings = model.encode(texts, normalize_embeddings=True)
    return np.asarray(embeddings)

def search(model: SentenceTransformer, query: str, filenames: list[str],
           embeddings: np.ndarray, top_k: int = 3) -> list[tuple[str, float]]:
    query_vec = model.encode([query], normalize_embeddings=True)[0]
    scores = embeddings @ query_vec  # cosine similarity, since vectors are normalized
    top_indices = np.argsort(-scores)[:top_k]
    return [(filenames[i], float(scores[i])) for i in top_indices]
```

Because embeddings are normalized, the dot product `embeddings @ query_vec` *is* cosine similarity — no separate normalization step needed at query time.

Run this against all ten sample reports with `query="stuck pipe"` and `top_k=10` (the whole ranking, not just the top few), and you get the real numbers behind the callout above:

```
1. 0.2239  Completion_003_2021-01-06
2. 0.2076  Drilling_038_2020-11-26   <- the actual stuck-pipe day
3. 0.1731  Drilling_049_2020-12-07
4. 0.1635  Drilling_003_2020-10-22
5. 0.1496  Drilling_039_2020-11-27   <- tight hole / high torque, findable now
6. 0.1495  Drilling_048_2020-12-06
7. 0.1458  Drilling_050_2020-12-08
8. 0.1437  Drilling_019_2020-11-07
9. 0.1366  Drilling_037_2020-11-25
10. 0.1319 Drilling_036_2020-11-24
```

Report #39 went from *absent* (Chapter 3) to *rank 5 of 10* — a real improvement, worth having. But it's not a clean win: the scores are bunched close together (0.13–0.22), and a busy engineer scanning only the top 2 or 3 results would still miss it.

## 4.6. Field notes

⚠ Field notes: why whole-document embeddings are noisy

**Query:** `"stuck pipe"`, same as above.
**Result:** report #39 ranks 5th of 10 when you embed each *entire report* as one vector — an improvement over keyword search's zero, but not a clean fix.
**Why:** each report is one page, but that page holds seven distinct data tables (casing, mud, drill bits, pumps, BHA, survey data, consumables) plus the narrative time breakdown. Embedding the whole thing blends a handful of sentences of relevant narrative with a few hundred words of numeric table content. The relevant signal — "Work tight hole… high torque… pull out of hole" — is a small fraction of what actually goes into the vector.
Isolate just the narrative lines instead of the whole report, and the picture changes completely:

```
0.4868  report #38 - "Work pipe, circulate lube sweep, work tool
```

```
                      back in position, Pipe free"
0.3359  report #38 - "During the slide lost tool face and became
                      assembly became stuck"
0.2402  report #39 - "Due to high torque decision to pull out of hole"
0.2329  report #49 - "Attempted multiple times to set packers..."
0.2143  report #39 - "Work tight hole at 6,526 feet."
0.1892  report #39 - "Hole drag from 6,050' to 5,901' no issues"
0.1892  report #36 - "Trip out of hole with BHA #17 core assembly..."
        (genuinely unrelated coring operation, same score as above)
```

At line granularity, report #39's high-torque line clearly separates from genuinely unrelated content (0.24 vs. 0.19) — something whole-document embedding couldn't show at all.
**Lesson:** semantic search doesn't fail because the *technique* is wrong — it fails here because the *unit of retrieval* is too coarse. This is exactly the problem Chapter 7's chunking solves, and it's worth remembering the next time a semantic search "isn't working": check what's actually inside each embedded vector before blaming the model.

## 4.7. Practical exercise

**Try it yourself:** Embed all ten sample DDRs and run `search(model, "stuck pipe", filenames, embeddings, top_k=10)`.

**You'll know it worked when:** `FORGE-16A-78-32_Drilling_039_2020-11-27.txt` appears somewhere in your ranked results — unlike Chapter 3, where it didn't appear at all — even if it isn't near the top.

## 4.8. Challenge exercise

**Challenge:** Reproduce the Field Notes line-level result yourself. Manually pull three lines of text — report #39's `"Due to high torque decision to pull out of hole"`, report #39's `"Hole drag from 6,050' to 5,901' no issues"`, and report #36's `"Trip out of hole with BHA #17 core assembly, lay down core barrels"` (a genuinely unrelated line) — embed just those three strings, and confirm the high-torque line scores meaningfully higher against `"stuck pipe"` than the other two, which should score close to each other. A reference solution is in `code/chapter_04/challenge/`.

## 4.9. Key takeaways

- Embeddings represent meaning as geometry: similar meaning, nearby vectors.
- Cosine similarity is the standard comparison because it isolates direction (meaning) from magnitude.

- Semantic search over whole documents is a real improvement on keyword search — report #39 goes from unfindable to rank 5 of 10 — but "an improvement" and "solved" are different claims. Don't oversell it.
- Granularity is often the actual lever, not the embedding model. The same query, same model, same text — just isolated to individual narrative lines instead of a whole noisy report — turns a middling rank-5 result into a clear, checkable win.
- Brute-force cosine search (a matrix multiply) is entirely adequate at ten documents. It stops being adequate well before you reach the full 76-report archive's scale — that's Chapter 8's problem to solve.

## 4.10. Repository files

| File | Purpose |
|---|---|
| `code/chapter_04/semantic_search.py` | Embedding generation and brute-force cosine search |
| `notebooks/chapter_04_explore.ipynb` | Interactive companion notebook |

## 4.11. Suggested next step

You can now retrieve relevant passages by meaning, and you've learned the honest limits of doing it at whole-document granularity. Chapter 5 turns retrieval into an answer: given a question, retrieve the right evidence, and generate a response that cites exactly where each claim came from. Chapter 7 comes back to fix the granularity problem directly.

# 5. First RAG System

## 5.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain retrieval-augmented generation (RAG) as two separate, inspectable steps: retrieve, then generate.
- Assemble retrieved chunks into a prompt that forces a language model to answer only from the evidence it's given.
- Produce an answer with an explicit, per-claim evidence list — report number and page — rather than an unsupported paragraph.

## 5.2. Operational problem

Retrieval (Chapter 4) gets you the right passages. What an engineer actually wants is an answer: *"What led to the fishing operation on report #50?"* — not two ranked text chunks they have to read and synthesise themselves. But a generated answer that isn't traceable back to specific reports is worse than no answer at all in an engineering context, because it invites misplaced trust. This chapter builds the smallest system that does both: synthesise an answer, and show exactly where every part of it came from.

## 5.3. Example DDR extract

Target interaction, grounded in two real, adjacent reports

```
Question: What led to the fishing operation on report #50?

Answer: On report #49 (2020-12-07), the crew attempted multiple times
to set packers on BHA #32, but pressure readings showed the ball did
not seat and the packers failed to set. They tripped out with the
packer assembly and picked up a fishing BHA (#33) that same day. On
report #50 (2020-12-08), that fishing run milled up lost pieces of bit.

Evidence:
  FORGE-16A-78-32_Drilling_049_2020-12-07.pdf
  FORGE-16A-78-32_Drilling_050_2020-12-08.pdf
```

Both claims here are directly quoted from real report text — nothing in this answer was inferred beyond what the two reports state. This is the *target*. Keep reading — the practical exercise below checks whether the simple system this chapter builds actually reaches it.

## 5.4. Theory

**RAG (retrieval-augmented generation)** (Lewis et al. 2020) is exactly the two chapters you just built, chained together: retrieve relevant evidence (Chapter 4), then hand that evidence to a language model and ask it to answer *using only that evidence*. The model is not answering from what it memorised during training — it's answering from the specific passages you retrieved, which is what makes citation possible at all.

The critical design decision is the prompt: instruct the model explicitly to answer only from the provided context, and to cite which source each part of the answer came from. This doesn't make hallucination impossible — Chapter 10 covers stronger mitigations — but it is the foundation everything else builds on.

Here's the complete pipeline you've actually built, chapter by chapter — every arrow below is real, working code you've already written:

```
DDR PDF
   ↓
Extraction              (Chapter 1 - pdfplumber)
   ↓
Cleaning                (Chapter 2 - abbreviation expansion)
   ↓
Embeddings              (Chapter 4 - sentence-transformers)
   ↓
Retriever               (Chapter 4 - cosine similarity search)
   ↓
LLM                     (this chapter - prompt + generation)
   ↓
Answer + Citations
```

This is deliberately the simplest version of this pipeline that could possibly work — brute-force search, whole-document embeddings, no reranking, no traceability guardrails beyond a citation list. Part II rebuilds nearly every arrow in this diagram to survive a real, full-scale archive.

## 5.5. Implementation

```python
# code/chapter_05/first_rag.py
from pathlib import Path

from semantic_search import embed_texts, load_chunks, search  # from Chapter 4
```

```
PROMPT_TEMPLATE = """Answer the question using ONLY the evidence below.
If the evidence doesn't contain the answer, say so - do not guess.
Cite which report each part of your answer comes from.

Evidence:
{evidence}

Question: {question}
Answer:"""

def build_prompt(question: str, retrieved: list[tuple[str, float]],
                 filenames: list[str], texts: list[str]) -> str:
    evidence_blocks = []
    for filename, _score in retrieved:
        text = texts[filenames.index(filename)]
        evidence_blocks.append(f"[{filename}]\n{text}")
    return PROMPT_TEMPLATE.format(
        evidence="\n\n".join(evidence_blocks),
        question=question,
    )

def answer_question(question: str, model, filenames, texts, embeddings,
                    llm_call) -> tuple[str, list[str]]:
    retrieved = search(model, question, filenames, embeddings, top_k=3)
    prompt = build_prompt(question, retrieved, filenames, texts)
    answer_text = llm_call(prompt)  # plug in your LLM of choice
    evidence = [filename for filename, _score in retrieved]
    return answer_text, evidence
```

`llm_call` is intentionally a plain function argument, not a hardcoded API client: plug in a local model, an API-based model, or — while you're first testing the retrieval and prompt logic — a stub that just echoes the evidence back, so you can verify the pipeline end to end before spending a single API call.

## 5.6. Practical exercise

**Try it yourself:** Wire up `answer_question` with a stub `llm_call` that simply returns the evidence text, run it for "What led to the fishing operation on report #50?" with `top_k=3`, and print the evidence list.

**You'll know it worked when:** you can see exactly which three reports came back — and, before reading further, decide for yourself whether that list actually contains what the target answer above needs.

## 5.7. Field notes

⚠ Field notes: the report the question is about isn't in the evidence

**Query:** `"What led to the fishing operation on report #50?"`, `top_k=3`.
**Result:** retrieval returns report #49 (rank 1) plus two other reports — but **not report #50**, the report the question is literally about. The full ranking:

```
1. 0.4412  Drilling_049_2020-12-07   <- packers fail (correctly retrieved)
2. 0.3461  Drilling_038_2020-11-26
3. 0.3358  Drilling_037_2020-11-25
...
8. 0.3012  Drilling_050_2020-12-08   <- the fishing report itself
```

**Why:** report #50's own text — bit-mill fishing narrative buried among casing tables, mud tables, and a full BHA component list — is, at whole-document granularity, more similar to *several other reports'* tables than it is to a question about fishing. This is the same granularity problem Chapter 4 already flagged; here it has a sharper consequence.
**Lesson:** this is exactly the failure mode this chapter's own Key Takeaways warn about — and it's a dangerous one specifically *because* the question names "report #50" directly. A careless `llm_call` implementation could echo that report number back in its answer without ever having actually retrieved report #50's content, producing a citation-shaped sentence that *looks* grounded and isn't. Always check what's actually in the evidence list — not what the question happens to mention — before trusting an answer. Chapter 7's chunking and Chapter 9's hybrid retrieval are what actually close this gap; Part I's simple system doesn't, and pretending otherwise would defeat the entire point of this book.

## 5.8. Challenge exercise

**Challenge:** Connect a real LLM (local via `transformers`/`ollama`, or a hosted API) as `llm_call`, and add a check that rejects the answer if it mentions a report filename that wasn't in the retrieved evidence list — a first, crude hallucination guard. Then ask it a question the ten-report sample archive genuinely can't answer (e.g. "what happened on report #100?") and confirm it says so rather than inventing an answer. A reference solution is in `code/chapter_05/challenge/`.

## 5.9. Key takeaways

- RAG is retrieval, then generation — two separate, individually debuggable steps, not one opaque black box.
- If retrieval is wrong, generation cannot save you — always verify the evidence list before trusting the generated answer.
- A prompt that instructs "answer only from evidence, cite sources" is necessary but not sufficient for trustworthy answers. Part II builds the guardrails that make it sufficient.

## 5.10. Repository files

| File | Purpose |
|---|---|
| `code/chapter_05/first_rag.py` | Prompt assembly and evidence-tracked question answering |
| `notebooks/chapter_05_explore.ipynb` | Interactive companion notebook |

## 5.11. Suggested next step

What you've built in Part I works — on ten clean, digital, hand-picked reports out of the full 76-report Utah FORGE archive. Part II industrialises this system against the reality of a full archive: retrieval that has to be both fast and precise across every report, and answers that have to survive scrutiny from someone who wasn't in the room when they were generated. Chapter 6 starts with the first thing that breaks in a typical real-world archive: scanned reports with no extractable text at all.

# Part II.

# Part II — Industrialising the System

# 6. Scanned Reports and OCR Quality Gates

## 6.1. Learning objectives

By the end of this chapter, you will be able to:

- Recognise when a DDR PDF has no extractable text and needs OCR.
- Run OCR on a scanned page and understand why the raw output is often too noisy to index as-is.
- Build a quality gate that rejects bad OCR output *before* it enters your search index, rather than trying to fix it after the fact.

## 6.2. Operational problem

Every one of the 76 Utah FORGE reports used in this book is digital-native — the archive's QA/QC pass confirms 0 unparsed PDFs across 227 source files. You got lucky: WellEz, the software that generated these reports, embeds real character data on every page. Not every archive is this well-behaved. Faxed reports, reports scanned from paper files, and photos of a logbook page all produce PDFs with no character data at all — just a picture of text. Chapter 1's `extract_text()` handled this gracefully (an empty string, not a crash), but a real, larger archive needs more than graceful failure: it needs OCR to recover the text, and a way to know whether to trust what OCR gives back.

## 6.3. Example DDR extract

**i** OCR output before and after quality gating

```
Clean digital extraction (this is what every Utah FORGE report gives):
  "During the slide lost tool face and became assembly became stuck"

Degraded OCR output from a hypothetical poor scan of the same line:
  "Dur1ng the s1ide los+ t00l face and became assembly became stu(k"
```

The second block is exactly the kind of text a naive pipeline would embed and index without complaint — and exactly the kind of text a quality gate should catch and route to a review queue instead.

## 6.4. Theory

Getting OCR to run is the easy part — `pytesseract`, cloud OCR APIs, and PDF-rendering libraries all do a competent job of turning a page image into text. The hard part, and the one that actually determines whether your downstream system is trustworthy, is deciding **whether to trust the output at all**. A quality gate scores OCR text against a handful of cheap heuristics — is there enough text, is the alphabetic-to-symbol ratio sane, is there suspicious repeated garbage — and rejects anything that fails, routing it to manual review instead of silently indexing noise.

This chapter's companion pipeline, **DDR_UTAH_FORGE**, implements exactly this pattern in `src/rag_pdf/ocr_quality.py` — even though, as noted above, this particular archive never needs to exercise it. That's by design: the quality gate runs unconditionally on every extracted page, digital or scanned, as a defensive check. Its `evaluate_ocr_quality()` function checks four independent flags — `low_text_density`, `high_symbol_ratio`, `repeated_garbage`, `low_line_count` — and rejects a page only once **two or more** flags fire, so no single noisy-but-legitimate page (e.g. a short report with a lot of numeric data) gets discarded on one false positive.

```
extracted text
   ↓
compute four flags
(low_text_density, high_symbol_ratio,
 repeated_garbage, low_line_count)
   ↓
count how many are True
   ↓
2 or more active?
   ↓
  yes -> reject, flag for review
  no  -> accept into index
```

## 6.5. Implementation

Build a simplified version of the same idea:

```python
# code/chapter_06/ocr_quality_gate.py
import re

def evaluate_ocr_quality(text: str, min_chars: int = 200,
                         min_alpha_words: int = 30,
                         max_symbol_ratio: float = 0.35,
                         reject_min_flags: int = 2) -> dict:
    alpha_tokens = re.findall(r"[A-Za-z]+", text)
    non_ws = [c for c in text if not c.isspace()]
    noisy = sum(1 for c in non_ws if not c.isalnum() and c not in set("%.,()/-:'"))
    symbol_ratio = noisy / max(len(non_ws), 1)
```

```
    flags = {
        "low_text_density": len(text) < min_chars or len(alpha_tokens) < min_alpha_words,
        "high_symbol_ratio": symbol_ratio > max_symbol_ratio,
    }
    active = [k for k, v in flags.items() if v]
    return {"reject_ocr": len(active) >= reject_min_flags, "flags": flags}
```

Test it against a real Utah FORGE report's clean text (it should pass easily — that text has a healthy word count and almost no noise symbols) and against the corrupted example above (it should fail on both flags).

The full production version — including the repeated-garbage detector and line-count check — is in the companion repository at `src/rag_pdf/ocr_quality.py`, alongside `src/rag_pdf/rotation_handler.py` for detecting and correcting upside-down or sideways scans, and `src/rag_pdf/table_ocr_handoff.py` for the case where a *table* (not narrative text) needs OCR fallback.

## 6.6. Practical exercise

**Try it yourself:** Run `evaluate_ocr_quality()` against the extracted text of report #38 (clean) and the degraded example string above.

**You'll know it worked when:** the real report text passes cleanly, and you can explain, from the flags returned, exactly why the degraded text fails.

## 6.7. Field notes

Field notes: checking the gate doesn't cry wolf

**Action:** run `evaluate_ocr_quality()` against all ten real sample reports, including report #3 — the sparsest one in the archive, written before the well was even spudded, with most of its tables entirely empty.
**Result:** every single report passes cleanly. Report #3, the thinnest of the ten, still has 2,220 characters and 360 alphabetic words — comfortably clear of the 200-character / 30-word thresholds:

```
chars=2220 alpha_words=360 symbol_ratio=0.007  reject=False
```

**Why:** even a nearly-empty DDR carries a full page of section headers, column labels, and boilerplate ("WELL/JOB INFORMATION", "PRESENT OPERATIONS", table column names) regardless of how little actually happened that day. That structural text alone is enough to clear the density thresholds.
**Lesson:** a quality gate is only trustworthy if it doesn't also reject legitimate sparse content — a gate that flagged every quiet day as "probably OCR garbage" would be useless in practice, burying real pages under false alarms. Checking the gate against your *most* sparse real documents, not just your noisiest fake ones, is how you find that out before it costs you.

## 6.8. Challenge exercise

**Challenge:** This archive is 100% digital, so you can't test the gate against a real scanned Utah FORGE page. Instead, write a small function that synthetically corrupts clean text (swap random characters for lookalikes, inject repeated tokens) at increasing severity, and find the corruption level at which `evaluate_ocr_quality()` starts rejecting it. This is a legitimate way to stress-test a quality gate before you ever encounter the real noisy data it's meant to catch.

## 6.9. Key takeaways

- OCR quality gating is cheap, heuristic, and worth running unconditionally — even an archive that's currently 100% digital may not stay that way, and the check costs almost nothing on clean text.
- Require multiple independent signals to agree before rejecting a page; a single heuristic alone produces too many false positives.
- Reject-and-flag-for-review beats silently-index-anyway every time traceability matters more than completeness.

## 6.10. Repository files

| File | Purpose |
|---|---|
| `code/chapter_06/ocr_quality_gate.py` | Simplified OCR quality gate |
| `DDR_UTAH_FORGE/src/rag_pdf/ocr_quality.py` | Production quality gate (companion repo) |
| `DDR_UTAH_FORGE/src/rag_pdf/rotation_handler.py` | Scan rotation detection (companion repo) |

## 6.11. Suggested next step

Once you trust the text coming out of OCR and digital extraction alike, the next question is how to break it into pieces small enough to embed and retrieve — without cutting a stuck-pipe narrative in half across two chunks. That's Chapter 7.

# 7. Chunking That Respects Report Structure

## 7.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain why chunk size is measured in tokens, not characters or words.
- Implement token-based chunking with overlap using `tiktoken`.
- Implement segment-aware chunking that breaks at natural report boundaries (section headers) instead of at an arbitrary character count.

## 7.2. Operational problem

Chapters 1–5 embedded each report as a single unit — fine for a ten-report sample corpus, unworkable at the full archive's scale. A single Utah FORGE report already has seven distinct sections (WELL/JOB INFORMATION, BOP, CASING, MUD, DRILL BITS, PUMPS, BHA, SURVEY DATA, CONSUMABLES, TIME BREAKDOWN) packed onto one page. Embed too much of that in one vector and the embedding blurs the casing programme together with the stuck-pipe narrative, hurting retrieval precision. Split naively at a fixed character count and you'll cut report #38's stuck-pipe sentence in half mid-word, right across a chunk boundary — the exact passage you most need to retrieve becomes the two chunks least likely to score well.

## 7.3. Example DDR extract

**i** Why naive splitting fails, using real report #38 text

```
Naive fixed-character split, mid-sentence:

Chunk A: "...23:30 04:00 4.5 Drill From 6,360' to 6,507', (147') Total,
          32.6' feet per hour. WOB 20 TO 35k, Rotary 50, Torque 6,500,
          SPP 3200-3400 GPM 560, DIFF 200-300psi During the slide lost
          tool face and became assembly became st"
Chunk B: "uck 04:00 06:00 2.0 Work pipe, circulate lube sweep, work
          tool back in position Pipe free"
```

A query about "what happened when the assembly got stuck" now has its key evidence split across two separately-scored chunks — neither of which alone contains the complete sentence.

## 7.4. Theory

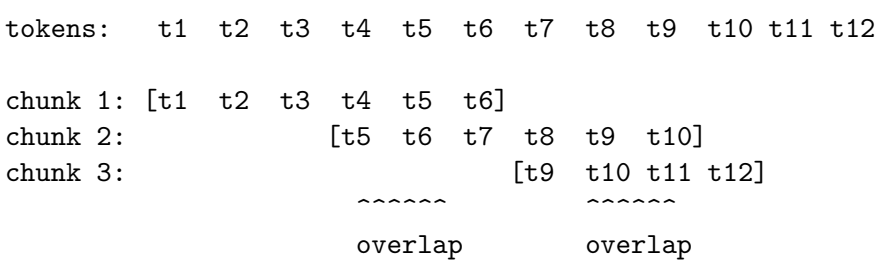
Two ideas, used together, fix this:

1. **Token-based sizing.** Language models and embedding models operate on tokens, not characters — and token count is what actually determines whether a chunk fits a model's context window and how much a given chunk "costs" downstream. Chunking by token count (with a library like `tiktoken`) is more accurate than chunking by character or word count.
2. **Segment-aware boundaries.** Rather than cutting at a fixed token count regardless of content, detect natural boundaries — section headers like `TIME BREAKDOWN` or `SURVEY DATA`, which every Utah FORGE report uses consistently — and prefer to split there. A chunk that ends at a genuine section break is far more likely to be a self-contained, retrievable unit of meaning than one that ends mid-sentence because a counter hit its limit.

The companion pipeline, **DDR_UTAH_FORGE**, implements both in `src/rag_pdf/chunking.py`. `chunk_text_by_tokens()` does token-based splitting with configurable overlap (so context isn't lost entirely at a boundary), falling back to a word-based approximation if `tiktoken` isn't available. `split_text_for_segment_aware_chunking()` goes further: it walks the text line by line, detects boundary patterns, and returns a list of `SegmentBlock` objects — each with a title, its text, and a `boundary_type` (`MATCH`, `INSERT`, or `CONTINUATION`) recording *why* the segment starts where it does.

Run against the real 76-report archive, this pipeline's chunking produces **2,943 chunks** — an average of about 39 chunks per report, reflecting just how much structured content (header, seven data tables, and a narrative time breakdown) each single-page DDR actually contains.

Token-based chunking with overlap looks like this — each chunk shares a few tokens with its neighbour, so no boundary loses context entirely:




## 7.5. Implementation

```python
# code/chapter_07/token_chunking.py
import tiktoken

def chunk_text_by_tokens(text: str, chunk_tokens: int = 200,
                         overlap_tokens: int = 40) -> list[str]:
    enc = tiktoken.get_encoding("cl100k_base")
    tokens = enc.encode(text.strip())
```

```
    chunks, start = [], 0
    while start < len(tokens):
        end = min(len(tokens), start + chunk_tokens)
        chunks.append(enc.decode(tokens[start:end]).strip())
        if end == len(tokens):
            break
        start = max(0, end - overlap_tokens)
    return chunks
```

This is a direct simplification of `chunk_text_by_tokens()` in the companion repository — same sliding-window-with-overlap logic, without the word-based fallback path. Read the full version, plus `split_text_for_segment_aware_chunking_with_patterns()`, in `src/rag_pdf/chunking.py`.

## 7.6. Practical exercise

**Try it yourself:** Chunk report #38's full text with `chunk_tokens=60, overlap_tokens=15`, and confirm the stuck-pipe sentence ("During the slide lost tool face and became assembly became stuck") appears complete within at least one chunk rather than split across two.

**You'll know it worked when:** you can point to a single chunk that contains the full sentence, from "During the slide" through "became stuck."

## 7.7. Field notes

Field notes: does the heading heuristic ever guess wrong?

**Action:** run the "all-caps line under 6 words = heading" rule against every line of all ten real reports, and list every match.
**Result:** every single match — across all ten reports — is a genuine section header: `BHA`, `BOP`, `CASING`, `CONSUMABLES`, `DRILL BITS`, `MUD`, `PUMPS`, `SURVEY DATA`, `TIME BREAKDOWN`, `WELL/JOB INFORMATION`, plus the completion report's `FLUID DATA`, `FRAC`, `PERFORATIONS`, `SAFETY`, and `TIME LOG`. Zero false positives — no data row anywhere in the archive happens to look like a heading.
**Why:** this isn't luck — it reflects something real about how DDR software generates these reports. Section headers are short, fixed, all-caps labels from a small controlled vocabulary; data rows always carry numbers, units, or lowercase narrative text that the pattern correctly excludes.
**Lesson:** a heuristic that "seems reasonable" is still a guess until you've run it against real data and checked every match by hand. This one happens to be exactly right on this archive — but that's a claim worth verifying yourself before trusting it on a different one, not an assumption to inherit.

## 7.8. Challenge exercise

**Challenge:** Implement a minimal segment-aware splitter that treats any all-caps line under 6 words (like `MUD`, `DRILL BITS`, `SURVEY DATA`, `TIME BREAKDOWN`) as a section heading and starts a new segment there. Run it against report #38's full text and confirm it produces one segment per section rather than one blended mass of casing, mud, and time-breakdown text. A reference solution — and the full production logic — is in `code/chapter_07/challenge/` and `DDR_UTAH_FORGE/src/rag_pdf/chunking.py` respectively.

## 7.9. Key takeaways

- Chunk size belongs in tokens, because that's the unit every downstream model actually consumes.
- Overlap prevents context from vanishing at a chunk boundary, at the cost of some redundancy in the index.
- Segment-aware boundaries produce chunks that are more likely to be self-contained, retrievable units — worth the extra complexity once reports have real internal structure, which every DDR in this archive does.

## 7.10. Repository files

| File | Purpose |
|---|---|
| `code/chapter_07/token_chunking.py` | Simplified token-based chunker |
| `DDR_UTAH_FORGE/src/rag_pdf/chunking.py` | Production token + segment-aware chunking (companion repo) |

## 7.11. Suggested next step

Well-formed chunks still need somewhere fast to live. Chapter 8 replaces Chapter 4's brute-force cosine search with a real vector database, and scales retrieval from ten documents to the full 76-report archive.

# 8. Vector Databases at Scale

## 8.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain why brute-force cosine search stops being adequate as a corpus grows, in concrete terms (memory and latency).
- Build and query a FAISS index over DDR chunk embeddings.
- Understand the difference between a per-document index and a campaign-wide global index, and when each is the right tool.

## 8.2. Operational problem

Chapter 4's `embeddings @ query_vec` brute-force search works fine over ten documents — it's a single matrix multiply. The full Utah FORGE archive is 76 reports. Run this book's companion pipeline's chunking and embedding steps against all of them and you get **2,943 chunks** — still small by industrial standards, but already the point where you want a persistent, queryable index rather than an in-memory NumPy array you rebuild every session. And a production system needs more than one query mode: per-document search (fast, scoped to one report) and campaign-wide search across every chunk are genuinely different operations.

## 8.3. Theory

A **vector database** (or vector index library, like FAISS (Johnson et al. 2019)) organises embeddings so that similarity search doesn't require comparing the query against every single vector. At this book's scale, even the simplest FAISS index — `IndexFlatIP` (flat, inner-product) — is the right choice: it's an *exact* nearest-neighbour search, mathematically equivalent to Chapter 4's brute-force approach, just implemented in optimised C++ with a persistent, loadable index file. (Larger-scale approximate indexes — IVF, HNSW — trade a small amount of accuracy for speed at millions of vectors; not a tradeoff this book's scale needs.)

Because embeddings are normalized (Chapter 4), inner product *is* cosine similarity — which is exactly why `IndexFlatIP` is the right index type rather than a distance-based one.

## 8.4. Building the real index

The companion pipeline, **DDR_UTAH_FORGE**, builds two tiers of index:

- **Per-document index** (`scripts/build_index.py`) — one FAISS index per DDR, used when a question is scoped to a specific report. All 76 reports already have this: chunks, embeddings, and a `faiss.index` file sit in `data/processed/<doc_id>/` for every report in the archive.
- **Global index** (`scripts/build_global_index.py`) — a single FAISS index over every chunk across every report, used for cross-document questions like "which reports involve fishing operations."

Running `build_global_index.py` against this archive's 76 pre-processed documents produces a real, verifiable result:

```
Found 76 docs with embeddings
Concatenating 2,943 chunks from 76 docs...
Building FAISS IndexFlatIP ((2943, 384))...
  faiss.index - 2,943 vectors × 384d
  elapsed: 2.65s
```

384 dimensions matches `all-MiniLM-L6-v2`, the same embedding model from Chapter 4 — this pipeline doesn't switch models between the toy version and the production one, it just changes how the resulting vectors are indexed and queried.

The two tiers branch from the same per-document work, then serve different questions:

```
76 DDR PDFs
   ↓
chunks + embeddings
(per report)
   ↓
built into two indexes:
   ↓
Per-doc FAISS index
(one per report)
-> search THIS report
   ↓
Global FAISS index
(2,943 chunks, all reports)
-> search ACROSS reports
```

## 8.5. Implementation

```python
# code/chapter_08/build_faiss_index.py
from pathlib import Path

import faiss
import numpy as np

def build_index(embeddings: np.ndarray) -> faiss.IndexFlatIP:
    dimension = embeddings.shape[1]
    index = faiss.IndexFlatIP(dimension)
    index.add(embeddings.astype("float32"))
    return index

def save_index(index: faiss.Index, path: Path) -> None:
    faiss.write_index(index, str(path))

def load_index(path: Path) -> faiss.Index:
    return faiss.read_index(str(path))

def search(index: faiss.Index, query_vec: np.ndarray, top_k: int = 5):
    scores, indices = index.search(query_vec.reshape(1, -1).astype("float32"), top_k)
    return list(zip(indices[0].tolist(), scores[0].tolist()))
```

This mirrors the indexing pattern in the companion pipeline's `src/rag_pdf/services/search_service.py`, which builds `IndexFlatIP` over chunk embeddings and persists it to `faiss.index` alongside each document's other artefacts.

## 8.6. Practical exercise

**Try it yourself:** Build a FAISS index over the ten sample DDR embeddings from Chapter 4, save it to disk, reload it in a fresh Python session, and confirm a query for "stuck pipe" returns the same ranked order as Chapter 4's brute-force search — report #38 at rank 2, report #39 at rank 5.

**You'll know it worked when:** the FAISS-based and brute-force results agree — because, mathematically, `IndexFlatIP` over normalized vectors *is* cosine similarity, just implemented differently.

## 8.7. Field notes

Field notes: proving 'exact, not approximate' rather than asserting it

**Action:** run five different real queries — `"stuck pipe"`, `"fishing operation"`, `"packers failed to set"`, `"torque problems"`, `"losses"` — through both the brute-force NumPy search and a FAISS `IndexFlatIP` built over the same ten embeddings, and compare the full ranked order each one returns.
**Result:** all five queries produce byte-for-byte identical rankings between the two methods.

```
'stuck pipe':              brute-force order == FAISS order: True
'fishing operation':       brute-force order == FAISS order: True
'packers failed to set':   brute-force order == FAISS order: True
'torque problems':         brute-force order == FAISS order: True
'losses':                  brute-force order == FAISS order: True
```

**Why:** this is exactly what the Theory section claimed — `IndexFlatIP` computes the *same* inner-product comparison as `embeddings @ query_vec`, just in optimised C++ rather than NumPy. There's no approximation step anywhere in this index type for it to disagree with brute force on.
**Lesson:** "mathematically equivalent" is a claim worth testing, not just trusting because it sounds right — especially before you rely on it at a scale where you can no longer eyeball the difference. It took five lines of code to confirm this one.

## 8.8. Challenge exercise

**Challenge:** If you have the full 76-report Utah FORGE archive (`datasets/forge_archive/`) and can run the heavier dependencies (`sentence-transformers`, `faiss-cpu`), chunk and embed all 76 reports yourself and confirm you land close to the real 2,943-chunk count above. Small differences are expected — your chunking parameters won't be identical to the production pipeline's — but you should be in the same order of magnitude. A reference solution is in `code/chapter_08/challenge/`.

## 8.9. Key takeaways

- `IndexFlatIP` at this book's scale is exact, not approximate — you're not trading away accuracy, only reimplementing brute-force search more efficiently and persistently.
- Per-document and global indexes serve genuinely different questions; build both rather than forcing one index to do both jobs.
- Scaling the *index* doesn't fix retrieval *quality* — that's the next chapter's problem.

## 8.10. Repository files

| File | Purpose |
|---|---|
| `code/chapter_08/build_faiss_index.py` | FAISS `IndexFlatIP` build/save/load/search |
| `code/chapter_08/build_full_archive.py` | Reproduces the full 76-report archive from the public source |
| `datasets/forge_archive/` | The full 76-report Utah FORGE archive |
| `DDR_UTAH_FORGE/scripts/build_index.py` | Per-document index builder (companion repo) |
| `DDR_UTAH_FORGE/scripts/build_global_index.py` | Campaign-wide global index builder (companion repo) |

## 8.11. Suggested next step

A fast index doesn't guarantee good retrieval. Chapter 9 looks at the retrieval scoring code itself — how sparse and dense signals get combined, and what it takes to make that combination numerically sound.

# 9. Hybrid Retrieval and Reranking

## 9.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain why semantic search alone underperforms on precise, entity- or number-heavy queries.
- Combine BM25 (sparse/exact) and dense (semantic) retrieval using Reciprocal Rank Fusion (RRF).
- Apply a cross-encoder to rerank a small candidate set for higher precision than either retrieval method alone.
- Read production retrieval-scoring code closely enough to explain *why* a specific formula is written the way it is, not just what it computes.

## 9.2. Operational problem

A query like *"which report had packers fail to set?"* has both a semantic component (packers, setting, failure — a topic) and effectively an exact-match component (you want report #49 specifically, not every report that mentions packers in passing). Dense embeddings capture the topical meaning well but blur precise distinctions; keyword search (Chapter 3) nails the exact terms but misses paraphrase entirely. Neither alone is enough — production retrieval systems combine both.

## 9.3. Theory

**BM25** (Robertson and Zaragoza 2009) scores a document by term frequency, adjusted by how rare each term is across the corpus (inverse document frequency, IDF) and normalised for document length. It's decades old, computationally cheap, and excellent at exact and near-exact matches — the opposite profile from dense embeddings.

**Reciprocal Rank Fusion (RRF)** combines two ranked lists by *rank*, not raw score: for each document, sum `weight / (k + rank)` across both lists. This sidesteps a real problem — BM25 scores and cosine similarities live on completely different, incomparable scales, so averaging raw scores directly would let whichever signal happens to have larger numbers dominate regardless of which is actually more informative.

```
Query
   ↓
scored two ways:
   ↓
```

```
BM25 (sparse, exact terms)
   ↓
Dense (semantic embedding)
   ↓
Reciprocal Rank Fusion
(combine by rank, not score)
   ↓
Cross-encoder rerank
(top candidates only)
   ↓
Results
```

**Cross-encoder reranking** is the last, most expensive step, and deliberately runs on only the top handful of fused candidates, not the whole corpus. Unlike a dense embedding (which encodes the query and each document *separately*, then compares vectors), a cross-encoder scores the query and a candidate passage *together*, letting it capture interactions between them that no fixed vector representation can. This makes it substantially slower per comparison — which is exactly why it's applied after fusion has already narrowed the field, not instead of it.

## 9.4. Reading the real fusion code

The companion pipeline's `src/rag_pdf/retrieval/hybrid_utils.py` and `src/rag_pdf/services/search_service.py` implement exactly this pipeline. Two details in the real code are worth understanding closely, because they're the kind of thing that looks like a stylistic choice until you see what breaks without it.

Detail 1: the IDF formula's shape isn't arbitrary

```
self.idf[term] = math.log(1.0 + ((self.n_docs - df + 0.5) / (df + 0.5)))
```

Compare this to the classic Robertson-Sparse-Selection IDF, `log((N - df + 0.5) / (df + 0.5))` *without* the `1.0 +`. For a term that appears in almost every document (`df` close to `n_docs`), the classic formula's ratio drops below 1 and the log goes **negative** — which then corrupts any downstream sum that assumes term contributions are non-negative. Wrapping the ratio as `1.0 + ratio` before taking the log guarantees the result stays non-negative for every possible `df`, because `1.0 + ratio` can never fall below 1. This single design choice removes an entire category of edge case, permanently, for a cost of one added constant.

Detail 2: normalization has to define its own tie-break

```
if hi <= lo:
    return {i: 0.0 for i in candidates}
```

Min-max normalization rescales scores into `[0, 1]` by subtracting the minimum and dividing by the range. If every candidate in a batch scores identically, `hi - lo` is zero — a division that, left unguarded, would raise or produce `NaN`. The guard above catches that case explicitly

> and returns a neutral value (0.0 for everyone) instead, so a tie in *this* signal doesn't crash the pipeline or corrupt the fused score — the other signal in the fusion is still free to discriminate between the tied candidates.

Neither of these edge cases shows up by running "more queries" against typical text — they only show up by reasoning about the math at its boundaries: what happens when a term is universal, what happens when every score ties. Retrieval scoring is numerical code, and it deserves the same edge-case discipline you'd apply to any other numerical code. The fusion weights themselves are also plain, readable constants in `search_service.py`: `RRF_K = 20`, `RRF_DENSE_WEIGHT = 0.5`, `RRF_BM25_WEIGHT = 2.0` — the exact-match signal is weighted four times higher than the semantic one in this pipeline's fused ranking, reflecting that for DDR text, exact-term matching often carries more precision than pure similarity.

## 9.5. Implementation

```python
# code/chapter_09/hybrid_search.py
from collections import defaultdict

def reciprocal_rank_fusion(ranked_lists: list[list[str]], k: int = 20,
                           weights: list[float] | None = None) -> list[tuple[str, float]]:
    weights = weights or [1.0] * len(ranked_lists)
    scores: dict[str, float] = defaultdict(float)
    for ranked_list, weight in zip(ranked_lists, weights):
        for rank, doc_id in enumerate(ranked_list, start=1):
            scores[doc_id] += weight * (1.0 / (k + rank))
    return sorted(scores.items(), key=lambda item: item[1], reverse=True)
```

## 9.6. Practical exercise

**Try it yourself:** Rank the ten sample DDRs for "packers fishing" using Chapter 3's keyword search and Chapter 4's semantic search separately, then fuse the two ranked lists with `reciprocal_rank_fusion(lists, k=20, weights=[2.0, 0.5])` — matching the companion pipeline's real weighting.

**You'll know it worked when:** report #49 (packers) and #50 (fishing) both rank near the top of the fused list, and you can explain why the weighting favours the keyword signal here.

## 9.7. Field notes

⚠ Field notes: hybrid fusion isn't automatically better — check both directions

**Query:** the same `"stuck pipe"` query from Chapters 3 and 4.
**Result:** with the pipeline's real fusion weights (BM25 × 2.0, dense × 0.5), report #39 ranks **9th of 10** — *worse* than Chapter 4's dense-only search, which ranked it 5th. Equal weighting (1.0 / 1.0) still only gets it to 7th.

```
BM25 alone:    report #39 ranks 9th of 10
Dense alone:   report #39 ranks 5th of 10   (Chapter 4)
RRF (2.0/0.5): report #39 ranks 9th of 10   (pipeline default weights)
RRF (1.0/1.0): report #39 ranks 7th of 10
```

Compare that to `"packers fishing"` (the practical exercise above), where fusion helps exactly as expected — report #49 stays 1st and report #50 jumps from 10th (dense alone) to 2nd (fused).
**Why:** BM25 has essentially nothing useful to say about report #39 for `"stuck pipe"` — it shares almost no vocabulary with the query, so BM25 ranks it 9th on its own. Fusing that weak, near-random signal in with a weight of `2.0` doesn't cancel out — it actively drags a mediocre dense result down further. For `"packers fishing"`, by contrast, both signals carry real information (the word "fishing" genuinely appears in report #50), so combining them helps.
**Lesson:** hybrid retrieval's benefit depends on *both* signals being informative for the query at hand — not on combining more signals being unconditionally better. A sparse signal with nothing to contribute isn't neutral when fused; it's noise with a weight attached, and can make a weak dense ranking worse. This is exactly why Chapter 11 insists on per-category evaluation rather than one aggregate score — "hybrid beats dense on average" can still be actively wrong for a specific, operationally important query like this one.

## 9.8. Challenge exercise

**Challenge:** Implement the two guards from the code study above against deliberately constructed edge cases: a term that appears in every sample document (test that your IDF stays non-negative), and a batch where every candidate scores identically (test that your normalization doesn't divide by zero). Confirm an unguarded version fails first. A reference solution is in `code/chapter_09/challenge/`.

## 9.9. Key takeaways

- Sparse and dense retrieval fail on different, complementary query types — combine them rather than picking one.
- Rank-based fusion (RRF) avoids the scale-mismatch problem that plagues naive score-averaging across different retrieval methods.

- Cross-encoder reranking buys precision at a real latency cost — apply it only to a small, already-fused candidate set.
- Retrieval scoring is numerical code. Boundary conditions — universal terms, tied scores — deserve explicit guards, and reading *why* a formula is shaped the way it is tells you more than reading what it computes.

## 9.10. Repository files

| File | Purpose |
|---|---|
| `code/chapter_09/hybrid_search.py` | Reciprocal Rank Fusion implementation |
| `DDR_UTAH_FORGE/src/rag_pdf/retrieval/hybrid_utils.py` | BM25 index, IDF formula, score fusion (companion repo) |
| `DDR_UTAH_FORGE/src/rag_pdf/retrieval/canonical_hybrid.py` | Full hybrid retrieval pipeline (companion repo) |
| `DDR_UTAH_FORGE/src/rag_pdf/services/search_service.py` | Cross-encoder rerank + fusion weights (companion repo) |

## 9.11. Suggested next step

Better retrieval still isn't the same as a trustworthy answer. Chapter 10 tackles the harder problem: making sure every generated claim traces back to real evidence, and that the system tells you honestly when it doesn't know.

# 10. Traceable Answers and Hallucination Mitigation

## 10.1. Learning objectives

By the end of this chapter, you will be able to:

- Distinguish questions that should be answered by generation from questions that should be answered by structured data lookup.
- Attach a verifiable citation (report number, page) to every claim a system produces.
- Detect and surface gaps in your own source archive, so the system's silence about a period isn't mistaken for "nothing happened."

## 10.2. Operational problem

Chapter 5 built a prompt that *asks* a language model to cite sources — necessary, but not sufficient. A model can still cite a plausible-sounding report number for a number it invented, especially for arithmetic questions ("how many hours of non-productive time in November?") that language models are statistically bad at answering exactly, no matter how good the prompt is. Separately, this archive — like any real archive — has a genuine gap, and a system that answers confidently from what it *has*, without flagging what it's *missing*, can quietly mislead an engineer who assumes silence means nothing happened.

## 10.3. Structured facts, already extracted

Language models can produce fluent, plausible claims that are simply false — a well-documented failure mode called hallucination (Ji et al. 2023). The single most effective mitigation in this book is a rule of thumb, not a model trick: **if a question can be answered by looking something up or computing it from structured data, don't generate the answer — compute it.**

Every report in this archive has already been parsed into a real, structured `ddr_facts.parquet` table by the companion pipeline. Here's an actual row from report #38 (2020-11-26):

A real row from ddr_facts.parquet, report #38

```
report_date: 2020-11-26        wellbore: FORGE-16A-78-32
start_time:  06:00              end_time:      11:00
duration_hr: 5.0                 phase:         Production Drilling
```

```
op_code:     Wire Line Logs      is_npt:        False
operation_text: "PJSM, pre job safety meeting. Rig up Schlumberger
                 run the UBI log from 5.200 to surface. Rig down
                 logging tools."
```

A question like *"how many hours did report #38 spend on wireline logging?"* has an exact answer sitting in this table — `duration_hr = 5.0` — computed once during extraction, not re-derived by a language model each time it's asked. Generation is reserved for what it's actually good at — narrative synthesis across multiple passages — and even then, every claim carries its source.

```
question
   ↓
answerable from structured data?
   ↓
yes -> query ddr_facts.parquet
    -> exact, re-derivable answer
   ↓
no  -> retrieve + generate (Ch.5)
    -> answer + citations
       (verify before trusting)
```

## 10.4. A real, honest gap in this archive

The second mitigation is **corpus-completeness awareness**. This archive has a genuine one: the last Drilling report is #76, dated **2021-01-03**. The Completion report (a DFIT — Diagnostic Fracture Injection Test) is dated **2021-01-06**. Two calendar days — January 4th and 5th — have no report of any kind. Every other day in the drilling programme, from spud on October 30, 2020 through report #76, has exactly one report, so this gap is real and specific to the Drilling-to-Completion handover, not an artefact of messy source data.

A RAG system only knows what's in its index. If you ask it "what happened on January 5th, 2021?" without gap-awareness, a poorly built system might either hallucinate an answer or simply retrieve the nearest chunks (report #76 or the Completion report) without ever telling you neither actually covers that date. Detecting the gap is a matter of checking the *sequence* of report numbers and dates for holes — not the content, just the presence.

## 10.5. Implementation

```python
# code/chapter_10/traceable_answers.py
from dataclasses import dataclass

@dataclass
class Citation:
    report: str
```

```
    page: int | None = None

def format_evidence(citations: list[Citation]) -> str:
    lines = [f"  {c.report}" + (f" page {c.page}" if c.page else "") for c in citations]
    return "Evidence:\n" + "\n".join(lines)

def find_date_gaps(report_dates: list[str]) -> list[tuple[str, str]]:
    """Return (start, end) date ranges with no report, given a sorted
    list of ISO date strings covering a continuous daily-reporting period."""
    from datetime import date, timedelta

    dates = sorted(date.fromisoformat(d) for d in report_dates)
    gaps = []
    for prev, curr in zip(dates, dates[1:]):
        missing_days = (curr - prev).days - 1
        if missing_days > 0:
            gaps.append(((prev + timedelta(days=1)).isoformat(), (curr - timedelta(days=1)).iso
    return gaps
```

## 10.6. Practical exercise

**Try it yourself:** Run `find_date_gaps()` against the report dates in this book's ten-report sample — `["2020-10-22", "2020-11-07", "2020-11-24", "2020-11-25", "2020-11-26", "2020-11-27", "2020-12-06", "2020-12-07", "2020-12-08", "2021-01-06"]`.

**You'll know it worked when:** the function reports several gaps — most of them simply reflecting that Part I's curated subset skips most of the archive's days on purpose. Then run it against the *full* 76-report archive's dates (`datasets/forge_archive/`) and confirm it finds exactly one real gap: January 4th–5th, 2021, right before the Completion report.

## 10.7. Field notes

⚠ Field notes: verify the automatic label, not just the automatic lookup

**Action:** check the real, computed `is_npt` flag — set once during extraction, not re-derived on every query — for every operations row on report #38, the stuck-pipe day.
**Result:** out of ten rows covering the full 24 hours, exactly **one** is flagged `is_npt = True`:

```
23:30-04:00  is_npt=True   "Drill From 6,360' to 6,507'... During the
                            slide lost tool face and became assembly
                            became stuck"
04:00-06:00  is_npt=False  "Work pipe, circulate lube sweep, work tool
                            back in position, Pipe free"
```

**Why:** the classifier flagged the 4.5-hour block where the pipe *became* stuck, but not the

following 2-hour block where the crew actually recovered it. That's a defensible reading — the drilling operation is what got interrupted — but it's also a real judgment call: an engineer asking "how many hours did this stuck-pipe event cost?" would probably expect all 6.5 hours counted, not 4.5.
**Lesson:** a structured field being *automatically extracted* doesn't mean it's automatically *correct for your question.* Trusting `ddr_facts.parquet` blindly here would silently undercount the incident by 2 hours, every time someone asks. The fix isn't to distrust structured data — Chapter 10's whole argument is that it beats generation — it's to read the classification logic once, understand exactly what it counts, and know the edge it sits on before you build a report or a chart on top of it.

## 10.8. Challenge exercise

**Challenge:** Extend `find_date_gaps()` to classify each gap's severity (e.g. a gap under 3 days is Low; a gap that crosses a report-type boundary — Drilling to Completion, as this one does — is High regardless of length, since it likely represents an unrecorded operational transition). A reference solution is in `code/chapter_10/challenge/`.

## 10.9. Key takeaways

- Prefer structured lookup over generation whenever the question is answerable from data you already extracted — it's exact, not just plausible.
- Every generated claim needs a citation an engineer can independently check, not just a citation-shaped sentence.
- A system that can tell you what it *doesn't have* is more trustworthy than one that answers fluently regardless of coverage — this archive's real two-day gap at the Drilling-to-Completion handover is exactly the kind of silence worth surfacing explicitly, not filling in.

## 10.10. Repository files

| File | Purpose |
|---|---|
| `code/chapter_10/traceable_answers.py` | Citation formatting and date-gap detection |
| `DDR_UTAH_FORGE/data/processed/qc/raw_pdf_missing_reports.csv` | Real, computed gap-detection output (companion repo) |

## 10.11. Suggested next step

You now have a system that retrieves well and answers honestly. Chapter 11 asks the question every system like this eventually needs answered with numbers, not impressions: *how good is it, really* — and where specifically does it still fail?

# 11. Evaluating Retrieval Quality

## 11.1. Learning objectives

By the end of this chapter, you will be able to:

- Build a hand-labeled evaluation set that records, per question, the expected source report and page.
- Compute `recall@k`, Mean Reciprocal Rank (MRR), and `NDCG@k`, and explain what each number actually tells you.
- Break results down by question category and difficulty to find *where* a system fails, not just *whether* it does.

## 11.2. Operational problem

"It feels like it's working" is not a standard you should ship an engineering decision-support tool against. Before this system goes in front of drilling engineers, you need a defensible answer to "how good is it" — and, more usefully, "which kinds of questions does it get wrong." The companion pipeline ships an evaluation harness for exactly this, `scripts/run_eval.py`, built against a real question set for a different (private, North Sea) campaign it was originally developed on. It has **not yet been run against the Utah FORGE archive** — no evaluation question set exists for this well yet. That's not a gap in the tooling; it's the natural next step, and this chapter walks you through doing it for real, against the same archive you've used throughout this book.

## 11.3. Building your own question set

Rather than presenting borrowed results, build a small evaluation set against the ten-report Part I sample, where you already know the ground truth from having read the reports yourself in Chapters 1–5:

> **i** A starter evaluation set for the Part I sample

```
Q1: "Which report describes pipe becoming stuck during a slide?"
    expected: FORGE-16A-78-32_Drilling_038_2020-11-26.pdf

Q2: "Which report shows packers failing to set?"
    expected: FORGE-16A-78-32_Drilling_049_2020-12-07.pdf

Q3: "Which report describes milling up lost pieces of bit?"
```

```
    expected: FORGE-16A-78-32_Drilling_050_2020-12-08.pdf

Q4: "Which report mentions mud fluid seepage losses?"
    expected: FORGE-16A-78-32_Drilling_019_2020-11-07.pdf

Q5: "Which report shows the crew rigging up before spud?"
    expected: FORGE-16A-78-32_Drilling_003_2020-10-22.pdf
```

Each question has exactly one verifiable correct answer, because you picked these five reports for their distinct, checkable events in Chapters 1–5. This is the right way to start an evaluation set: begin with cases you can verify by eye, before scaling to enough volume that you can no longer read every answer yourself.

## 11.4. Theory

Three metrics, each answering a slightly different question:

- **`recall@k`** — did the correct answer appear *anywhere* in the top k results? Simple, and the right first metric to compute.
- **Mean Reciprocal Rank (MRR)** — averages `1 / rank` of the correct answer across all questions. Rewards getting the right answer to rank #1, not just somewhere in the top 10 — closer to what a user actually experiences.
- **`NDCG@k`** (Normalized Discounted Cumulative Gain) — like recall, but weights a hit at rank 1 more than a hit at rank 5, and normalizes so the score is comparable across queries with different numbers of correct answers.

None of these require deep statistics to use well — they require a good evaluation *set*: real questions, with a real, known correct source recorded in advance. Building that set by hand, from your own archive, is more valuable than the metric computation itself, because writing the questions forces you to think like the engineer who will actually use the system.

```
hand-labeled questions
(question -> expected report)
   ↓
run each through the retrieval pipeline
   ↓
record the rank of the expected report
   ↓
compute recall@k, MRR, NDCG@k
   ↓
break results down by category and difficulty
   ↓
find the weakest category -> that's where to improve next
```

## 11.5. Implementation

```python
# code/chapter_11/eval_metrics.py
def recall_at_k(results: list[dict], k: int) -> float:
    hits = [r for r in results if r.get("rank") is not None and r["rank"] <= k]
    return len(hits) / len(results) if results else 0.0

def mrr(results: list[dict]) -> float:
    scores = [1.0 / r["rank"] if r.get("rank") else 0.0 for r in results]
    return sum(scores) / len(scores) if scores else 0.0

def ndcg_at_k(results: list[dict], k: int) -> float:
    import math
    scores = []
    for r in results:
        rank = r.get("rank")
        scores.append(1.0 / math.log2(rank + 1) if rank and rank <= k else 0.0)
    return sum(scores) / len(scores) if scores else 0.0
```

Each `result` dict records, per evaluation question, the rank at which the known-correct report was retrieved (or `None` if it wasn't in the top k considered). This mirrors the shape of `_recall_at_k()`, `_mrr()`, and `_ndcg_at_k()` in the companion repository's `scripts/run_eval.py`.

## 11.6. Practical exercise

**Try it yourself:** Run the five questions above through Chapter 9's hybrid search over the ten-report sample archive, record each correct report's rank, and compute `recall@3` by hand.

**You'll know it worked when:** you have a small CSV or JSON file mapping question → expected report → observed rank, and a `recall@3` score you can quote and defend — because you wrote and verified every question yourself.

## 11.7. Field notes

> ! Field notes: 'better' isn't monotonic — track one hard question across every chapter
>
> **Action:** track report #39's rank for the query `"stuck pipe"` through every retrieval method this book has built so far, instead of just looking at one final aggregate score.
> **Result:**
>
> ```
> Chapter 3  keyword (AND)         not found at all   (0 of 10)
> Chapter 4  dense, whole-document  rank 5 of 10
> Chapter 9  hybrid, pipeline weights (BM25x2.0/dense x0.5)  rank 9 of 10
> Chapter 9  hybrid, equal weights (1.0/1.0)                 rank 7 of 10
> ```

**Why:** each chapter's technique is a genuine improvement *in general* — that's not in question. But this specific, operationally important question doesn't improve monotonically as the pipeline gets more sophisticated. It gets a little better (Chapter 4), then gets worse twice in a row (Chapter 9's two weightings) before hybrid retrieval's other strengths (Chapter 9's `"pack ers fishing"` example) show up clearly.
**Lesson:** a single aggregate metric — `recall@5` averaged across your whole evaluation set — can climb steadily from chapter to chapter while a specific, real question you care about gets worse. This is exactly why Chapter 9 insisted on per-category breakdowns and why this chapter opened by building your own question set rather than trusting someone else's summary number: the only way to know if *your* hard questions are improving is to track them individually, the way this table just did.

## 11.8. Challenge exercise

**Challenge:** Extend your five-question set to at least 15 questions against the *full* 76-report archive (`datasets/forge_archive/`), and compute `recall@5`. This is genuinely useful, unpublished work: as far as this book's companion pipeline is concerned, you'll be building the first real evaluation set for Utah FORGE. Save your question set alongside your results — it's the seed of exactly the kind of evaluation harness `scripts/run_eval.py` runs at scale once one exists. A reference solution approach is in `code/chapter_11/challenge/`.

## 11.9. Key takeaways

- An evaluation set with known correct answers is more valuable than any single retrieval technique — it's what tells you whether a technique helped.
- `recall@k`, MRR, and `NDCG@k` answer different questions; report more than one.
- Don't trust a system's evaluation results from a different dataset as a proxy for how it performs on yours — a harness that scored well on one campaign says nothing certain about a new well until you build and run a question set against that well's own archive, which is exactly what this chapter had you do.

## 11.10. Repository files

| File | Purpose |
|---|---|
| `code/chapter_11/eval_metrics.py` | `recall@k`, MRR, `NDCG@k` implementations |
| `DDR_UTAH_FORGE/scripts/run_eval.py` | Evaluation harness (companion repo) — ready to point at a Utah FORGE question set once you build one |

## 11.11. Suggested next step

Every discipline from Chapters 6–11 — OCR gating, structured chunking, hybrid retrieval, traceability, evaluation — exists in service of a single payoff: finding things in an archive a human reading it linearly would miss. Chapter 12 builds toward that payoff, using a real sequence of events already in this archive.

# 12. Sequence Detection: Building Toward Cross-Well Intelligence

## 12.1. Learning objectives

By the end of this chapter, you will be able to:

- Combine structured numeric trends (torque, ROP) with narrative operation text to build a single, checkable operational story.
- Explain how a windowed leading-indicator detector works — and verify, from real thresholds in production code, exactly what "causal" and "escalating" mean before trusting either label.
- Understand honestly what this book's single-well archive can and can't demonstrate, and what it would take to scale this technique to genuine cross-well intelligence.

## 12.2. Operational problem

This is the payoff question every previous chapter has been building toward: *did something earlier predict trouble later?* A human reading 76 reports in sequence can, with effort, spot a pattern — but a system that has structured, classified, and indexed the same text can hold every report in mind at once, and check numeric trends a human would have to read a table on every single page to notice.

This chapter is deliberately honest about scale: Utah FORGE's public archive is **one well, one continuous drilling programme** — not the multi-well, multi-phase campaign this book's companion pipeline was originally built to analyse. So instead of presenting a large statistical finding, this chapter shows you exactly how to check a real, small pattern by hand, using the same technique — and shows you precisely what production code exists to scale that technique up once you have more than one well's worth of data.

## 12.3. A real, checkable pattern

Report #38's `TORQUE` header field — a single number recorded every day regardless of what happens — tells a real story across three consecutive days:

> ℹ Real header data, reports #36-38

```
Report #36 (2020-11-24): TORQUE: 4,400
Report #37 (2020-11-25): TORQUE: 4,200
```

```
Report #38 (2020-11-26): TORQUE: 5,615 (header) / 6,500 (during the slide)
                          "During the slide lost tool face and became
                           assembly became stuck"
```

Torque was flat — even slightly down — for two days, then jumped by roughly a third on the same day the assembly became stuck. This is a same-day correlation between a structured number and a narrative event, not a multi-day advance warning — and that distinction matters. Don't claim a leading indicator you haven't actually verified extends before the event; this pattern says "torque and the stuck-pipe event moved together," not "torque predicted the stuck pipe two days out."

Chapter 5's other real sequence — report #49's packers failing to set, followed same-day by picking up a fishing BHA, followed by report #50's actual fishing operation — is a genuine two-report causal chain, directly traceable and already fully verified with real text.

## 12.4. Theory: how the production tool would check this at scale

The companion pipeline's `src/ddr_rag/causality_analyzer.py` implements exactly this kind of check, generalized: for a given term, it compares frequency in a window before and after a boundary date, and labels a term **causal** only above a defined post-boundary NPT rate, and **escalating** only above a defined frequency-increase ratio. The real thresholds, read directly from the module:

```
TRANSITION_WINDOW_DAYS = 14   # days before/after a boundary to compare
MIN_TERM_FREQ = 3             # minimum occurrences before a term counts at all
NPT_SIGNAL_THRESHOLD = 0.40   # post-boundary NPT rate to call a term "causal"
ESCALATION_FREQ_RATIO = 1.5   # frequency increase to call a term "escalating"
```

Every one of these is a plain, readable number you can inspect, change, and justify — not a weight buried inside a trained model.

**! A real limitation worth understanding, not hiding**

This module's default phase configuration — `["MIRU", "COND1", "INTRM1", "INTRM2", "PROD1", "COMPZN"]` — comes from `operator_alpha`, a different (private, North Sea) campaign's phase vocabulary. Utah FORGE's real data doesn't have that phase structure: its `ddr_facts.parquet` phase field is `"Production Drilling"` for essentially the entire programme, right up to the Completion-DFIT report. Running `causality_analyzer.py` against this archive as-is would compare windows around phase boundaries that don't meaningfully exist in this well's data — the code would run, but the output wouldn't mean anything useful yet.
This is a genuinely common situation in real engineering work: code built and validated on one dataset doesn't automatically transfer to another just because the file paths line up. Before trusting output from an adapted pipeline, check that the assumptions baked into its configuration (here, a phase vocabulary) actually match your data. The `src/ddr_rag/npt_classifier.py` module, by contrast, *has* been adapted for Utah FORGE specifically — `classify_utah_forge_npt()` exists and recognises this archive's real fields — which tells you adaptation here is partial, not absent: some layers of this pipeline are ready for this well's data, and some aren't yet.

## 12.5. Implementation

```
# code/chapter_12/torque_trend_check.py
def torque_trend(readings: list[tuple[str, float]]) -> list[tuple[str, float, float]]:
    """Given [(date, torque_value), ...] in chronological order, return
    each day's percentage change from the previous day."""
    trend = []
    for (prev_date, prev_val), (date, val) in zip(readings, readings[1:]):
        pct_change = (val - prev_val) / prev_val if prev_val else 0.0
        trend.append((date, val, pct_change))
    return trend
```

```
readings = [
    ("2020-11-24", 4400),
    ("2020-11-25", 4200),
    ("2020-11-26", 5615),
]
for date, val, pct in torque_trend(readings):
    print(f"{date}: {val} ({pct:+.1%})")
# 2020-11-25: 4200 (-4.5%)
# 2020-11-26: 5615 (+33.7%)
```

## 12.6. Practical exercise

**Try it yourself:** Extend `readings` with report #39's header torque value (check the PDF yourself — Chapter 1's `read_ddr.py` will get it for you) and confirm whether the elevated torque persisted after the pipe was freed, or dropped back toward the pre-event baseline.

**You'll know it worked when:** you can state the real percentage change for each day, sourced from header fields you extracted yourself, not from this chapter's text.

## 12.7. Challenge exercise

**Challenge:** Pull the `TORQUE` header value for all 76 reports in `datasets/forge_archive/` (Chapter 1's extraction plus a small regex), plot it as a time series, and identify every day where torque jumps more than 25% from the previous day. Cross-reference each jump against that day's narrative text — how many of them correspond to a real operational event like report #38's, and how many are unremarkable? A reference solution is in `code/chapter_12/challenge/`.

## 12.8. Key takeaways

- A single structured number, tracked day over day, can corroborate a narrative event — but be precise about what you've actually shown: same-day correlation is not the same claim as

a multi-day leading indicator, and this chapter deliberately didn't claim more than the real data supports.
- Production causality-detection code carries real, inspectable thresholds — read them before trusting output, and check that its configuration (like a phase vocabulary) actually matches your dataset before running it.
- Genuine cross-well intelligence needs more than one well's worth of data to compare against — Utah FORGE's `data/fields/UtahForge/wells/` structure in the companion pipeline is already organized to support additional wells as this research programme (or your own multi-well archive) grows.

## 12.9. Repository files

| File | Purpose |
|---|---|
| `code/chapter_12/torque_trend_check.py` | Day-over-day trend check on a structured header field |
| `DDR_UTAH_FORGE/src/ddr_rag/causality_analyzer.py` | Windowed leading-indicator detection (companion repo) |
| `DDR_UTAH_FORGE/src/ddr_rag/npt_classifier.py` | Utah-FORGE-adapted NPT classification (companion repo) |
| `DDR_UTAH_FORGE/data/fields/UtahForge/wells/` | Multi-well data structure, ready for expansion (companion repo) |

## 12.10. The complete system

Twelve chapters ago, this was a single PDF and a five-line function. Here's everything you built, end to end — every arrow below is a chapter you completed, not an aspiration:

```
DDR PDF (scanned or digital)
   ↓
OCR quality gate                (Chapter 6)
   ↓
Extraction                      (Chapter 1)
   ↓
Cleaning                        (Chapter 2)
   ↓
Segment-aware chunking          (Chapter 7)
   ↓
Embeddings                      (Chapter 4)
   ↓
Hybrid index - BM25 + FAISS     (Chapters 8-9)
   ↓
Retriever + Reranker            (Chapter 9)
   ↓
```

```
LLM + structured facts          (Chapter 5, 10)
   ↓
Answer + Citations + Gaps flagged   (Chapter 10)
```

Two more pieces sit alongside this pipeline rather than inside it — they don't run on every query, but they're what make the rest of it trustworthy at scale:

```
Evaluation (Chapter 11) ---------- measures how well the pipeline above
                                    actually performs, against questions
                                    with known correct answers

Sequence detection (Chapter 12) -- mines what the pipeline has already
                                    indexed for patterns no single query
                                    would surface on its own
```

Every stage exists because an earlier, simpler version of this system failed at it — a scanned page with no text, a chunk that split a stuck-pipe sentence in half, a query that missed a report because it used the wrong three words, a hallucinated number where a table lookup belonged. None of the complexity here is complexity for its own sake.

## 12.11. Suggested next step

You've now built, chapter by chapter, a working RAG system grounded entirely in real, public DDRs — from extracting a single PDF's text through checking a real numeric trend against a real narrative event. Appendix A shows how to set up the full companion pipeline and point it at your own DDR archive, whether that's more Utah FORGE data as it becomes available, or a different well entirely.

# Part III.

# Appendices

# Appendix A: Environment Setup

This appendix covers everything you need before Chapter 1: installing dependencies, rendering the book, and understanding how Chapters 6–12 relate to the companion DDR Intelligence Platform pipeline.

## 1. Prerequisites

- Python 3.11 or later
- Quarto 1.7 or later (`quarto --version` to check)
- Git
- Positron (recommended) or any editor with a Python and Quarto extension

## 2. Set up the book's Python environment

From the `book/` directory:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Or, if you prefer conda/mamba:

```
conda env create -f environment.yml
conda activate ddr-rag-book
```

## 3. The sample dataset

Chapters 1–5 use ten real, curated Daily Drilling Reports from the public Utah FORGE archive (well FORGE 16A(78)-32) — already committed as PDFs in `datasets/sample_ddrs/`, since this is public data with no confidentiality concerns. Part II's "at scale" chapters use the full 76-report archive, committed at `datasets/forge_archive/`. Nothing needs to be generated or downloaded to start: `git clone` gives you the full dataset.

If you want to reproduce either curated subset from your own copy of the full public archive, or extend it with more reports, see `code/chapter_01/build_sample_archive.py` and `code/chapter_08/build_full_archive.py`.

## 4. Render the book

```
quarto render
```

Output is written to `_book/`. For live-editing a single chapter while you write:

```
quarto preview chapters/chapter_01.qmd
```

## 5. Working in Positron

Positron runs Quarto documents natively — open any `.qmd` file and use the **Render** button in the editor toolbar, or run individual code cells interactively (each ```` ```{python} ```` block behaves like a Jupyter cell). Recommended setup:

1. Open the `book/` folder as your Positron workspace root.
2. Select the `.venv` interpreter created in Step 2 via the interpreter picker (bottom status bar).
3. Use the **Quarto: Preview** command for live reload while editing a chapter.
4. Use the **Console** pane to run chapter scripts interactively (`python code/chapter_01/read_ddr.py ...`) — this is the fastest way to experiment with the code before committing it to a `.qmd`.

## 6. The companion pipeline (for Part II)

Chapters 6–12 reference **DDR_UTAH_FORGE**, a real, working DDR intelligence pipeline built specifically against this book's public archive — same architecture as the reference platform this project grew from (per-document extraction, hybrid retrieval, structured facts, causality analysis), adapted for Utah FORGE's real filenames and data. It's a separate repository — clone it alongside this book's repository, not inside it:

```
cd ..                       # up from book/ to your projects root
git clone https://github.com/djimrastephane/DDR_UTAH_FORGE.git
cd DDR_UTAH_FORGE
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

Its own `data/` folder is gitignored — the repository is code only. Copy this book's `datasets/forge_archive/` into `DDR_UTAH_FORGE/data/raw/` (or point it at your own copy of the public Utah FORGE archive) and follow its `README.md` Quickstart to run the full pipeline and Streamlit dashboard.

> **You don't need it to follow along**
>
> Part II chapters are written so every technique works fully standalone: the simplified code in `code/chapter_06/` through `code/chapter_12/` in *this* book's repository requires no access to the companion pipeline. Where a chapter cites a specific file or a specific result from `DDR_UTAH_FORGE` (for example, the real 2,943-chunk global index in Chapter 8, or the real report-gap dates in Chapter 10), that result was independently verified against the pipeline's actual output before being written into the book, so you can trust the numbers even without running the pipeline yourself.

## 7. A note on data handling

This book's own data (`datasets/sample_ddrs/`, `datasets/forge_archive/`) is public and safe to commit, share, and publish — that's true throughout this repository. The moment you point this book's code at your own organisation's DDRs instead, that changes:

- Never commit real, confidential DDR PDFs, extracted text, or derived artefacts (embeddings, indexes) to a public repository.
- If you're experimenting on a shared machine, confirm your organisation's data classification policy covers running third-party Python packages (embedding models, OCR engines) against confidential well data.
- Add your own archive's path to `.gitignore` before dropping any confidential PDFs into a local `datasets/` folder for testing.

## 8. Troubleshooting

| Symptom | Likely cause |
|---|---|
| `quarto render` fails on a code cell | Activate `.venv` before rendering — Quarto uses whichever Python is on `PATH`. Confirm with `which python` and `python -c "import pdfplumber"`. |
| `ModuleNotFoundError: sentence_transformers` (Chapter 4+) | Re-run `pip install -r requirements.txt`; this is a heavier dependency and sometimes needs a separate `pip install torch` first on some platforms. |
| FAISS import error on Apple Silicon (Chapter 8+) | Ensure you installed `faiss-cpu`, not `faiss-gpu` — the latter has no macOS wheel. |
| `read_ddr.py` reports no text extracted | Confirm you're pointing at a file in `datasets/sample_ddrs/` or `datasets/forge_archive/` — both are committed directly, so this shouldn't happen unless the path is wrong. |

# Appendix B: Oilfield Abbreviation Glossary

This glossary supports Chapter 2's abbreviation expansion engine and the Chapter 2 challenge exercise. It is broader than the `ABBREVIATIONS` dict built in Chapter 2 itself — that chapter's dictionary is deliberately scoped to terms verified present in this book's real Utah FORGE archive. This appendix adds enough additional common oilfield terms to give the expander real coverage on a first pass against a different archive. It is still not exhaustive — abbreviation usage varies by operator, rig contractor, and basin.

Terms marked **(FORGE)** are confirmed present in this book's real sample archive; the rest are common industry terms not found in this particular dataset but likely to appear in others.

| Abbreviation | Expansion |
|---|---|
| AFE | authorization for expenditure (FORGE) |
| BHA | bottom hole assembly (FORGE) |
| BOP | blowout preventer |
| CBL | cement bond log |
| CIRC | circulate |
| COND | condition |
| CSG | casing |
| DFIT | diagnostic fracture injection test (FORGE) |
| DFS | days from spud (FORGE) |
| DHSV | downhole safety valve |
| DLS | dogleg severity (FORGE) |
| DOL | days on location (FORGE) |
| DP | drill pipe |
| ECD | equivalent circulating density (FORGE) |
| EMW | equivalent mud weight |
| FIT | formation integrity test |
| FR | from |
| GPM | gallons per minute (FORGE) |
| HWDP | heavy weight drill pipe (FORGE) |
| KOP | kick-off point |
| LOT | leak-off test |
| MD | measured depth (FORGE) |
| MIRU | move in, rig up |
| MW | mud weight |
| MWD | measurement while drilling (FORGE) |
| NMDC | non-magnetic drill collar (FORGE) |
| NPT | non-productive time (FORGE) |
| OBSD | observed |
| PBTD | plugged back total depth |

| Abbreviation | Expansion |
|---|---|
| PDC | polycrystalline diamond compact (bit type) (FORGE) |
| PJSM | pre-job safety meeting (FORGE) |
| POOH | pull out of hole |
| PPG | pounds per gallon |
| PU | pick up (pipe) |
| PUW | pick up weight |
| RIH | run in hole |
| RKB | rotary kelly bushing (FORGE) |
| ROP | rate of penetration (FORGE) |
| RPM | revolutions per minute (FORGE) |
| SICP | shut-in casing pressure |
| SIDPP | shut-in drill pipe pressure |
| SLM | slow (rate), or short for "slim hole" depending on context — verify against operator style guide |
| SPP | stand pipe pressure (FORGE) |
| STDS | stands (of drill pipe) (FORGE) |
| STK | stuck |
| TD | total depth |
| TIH | trip in hole |
| TOOH | trip out of hole |
| TVD | true vertical depth (FORGE) |
| UBHO | universal bottom hole orientation sub (FORGE) |
| UBI | ultrasonic borehole imager (wireline log) (FORGE) |
| WOB | weight on bit (FORGE) |
| WOC | wait on cement |
| WOW | wait on weather |
| XO | crossover (sub) |

> ⚠ **Abbreviations are not universal**
>
> The same three letters can mean different things across operators and basins — build your abbreviation dictionary from *your* archive's actual usage before trusting it against your own reports, and treat any ambiguous abbreviation (like `SLM` above) as a signal to check context rather than expand automatically.

# References


Ji, Ziwei, Nayeon Lee, Rita Frieske, et al. 2023. "Survey of Hallucination in Natural Language Generation." In *ACM Computing Surveys*, No. 12, vol. 55.

Johnson, Jeff, Matthijs Douze, and Hervé Jégou. 2019. *Billion-Scale Similarity Search with GPUs*. https://arxiv.org/abs/1702.08734.

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, et al. 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems* 33: 9459–74.

Matthes, Eric. 2023. *Python Crash Course: A Hands-on, Project-Based Introduction to Programming*. 3rd ed. No Starch Press.

Reimers, Nils, and Iryna Gurevych. 2019. "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks." *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.

Robertson, Stephen, and Hugo Zaragoza. 2009. "The Probabilistic Relevance Framework: BM25 and Beyond." *Foundations and Trends in Information Retrieval* 3: 333–89.

Society of Petroleum Engineers. 2022. "Digitalisation of Daily Drilling Reports: Challenges and Opportunities." *SPE Digital Transformation Series*.